

TABS: Transforming Automatically BPMN Models into Blockchain Smart Contracts

Peter Bodorik
Faculty of Computer Science
Computer Science
Halifax, Canada
Peter.Bodorik@dal.ca¹

Christian Liu
Faculty of Computer Science
Computer Science
Halifax, Canada
Chris.Liu@dal.ca

Dawn Jutla
Sobey School of Business
Saint Mary's University
Halifax, Canada
Dawn.Jutla@gmail.com

Abstract

Research on blockchains addresses multiple issues, with one being automated creation of smart contracts. Developing smart contract methods is more difficult than mainstream software development as the underlying blockchain infrastructure poses additional complexity. We report on a new approach to develop smart contracts with the objective to automate the process to increase developer efficiency and reduce risks of errors introduced by software developers. To support industry adoption, we use Business Process Model and Notation (BPMN) modeling to describe an application while targeting applications in the trade vertical. We describe a system that transforms a BPMN model into a multi-modal model that combines Discrete Event (DE) modeling for concurrency with Hierarchical State Machines (HSMs) to represent application functionality. Then, further transformations are used to transform the DE-HSM model into methods in smart contracts. The system lets the modeler decide which of the independent patterns should be transformed into methods of a separate smart contract that is deployed on a sidechain for the purpose of (i) reducing processing costs and/or (ii) providing privacy so that other participants in the smart contract do not have visibility into processing of the pattern. We also briefly describe a proof-of-concept tool we built to demonstrate the feasibility of our approach.

Keywords—Blockchain, Business Processes Modeling Notation (BPMN), Discrete Event (DE) Modeling, Finite State Machine (FSM), Hierarchical State Machines (HSM), Smart Contract, Off-chain computation, Privacy, Smart Contract Interoperability

I. INTRODUCTION

Blockchains have received fierce attention since the publication of the Bitcoin white paper in 2008 and the subsequent creation of the Bitcoin blockchain in 2009. Although most public attention focuses on cryptocurrency trading, business, research and software industry communities have espoused the technology due to its desirable properties of trust, immutability, availability, and transparency, amongst others. Thus, blockchain platforms, accompanying languages and support tools for writing smart contracts are deservedly getting increased attention. However, as with any new technology, blockchains and their smart contracts also pose new challenges, in both the blockchain infrastructure and blockchain application development, that have slowed down adoption of the technology.

Blockchain application scalability, transaction throughput, and high cost are some of the main issues that are being addressed by research. To avoid the high cost of storage and commutation, Eberhardt and Tai [1] discuss techniques that can be used for off-chain storage and computation to reduce their cost. They categorized methods based on what is moved to off-chain into: (i) off-chain storage, (ii) off-chain computation, and (iii) a hybrid approach, in which both storage and computation are off-chained. In [2], [3], computation is performed off-chain while proofing techniques are used to record the results of computation on the blockchain.

Another impediment to blockchain adoption is the high cost of the consensus algorithm used to ensure, by the majority of the blockchain network nodes, that a block of data to be appended to the chain is valid in face of errors or attacks. For example, Bitcoin's consensus algorithm, which uses a proof-of-work approach, results in a maximum of five to seven transactions per second, while the Ethereum's network consensus limits transactions to about 15 per second. Yang (2020) assessed the capacity of blockchain networks and showed that it is still a critical problem and new approaches, such as those utilizing shards [3], [4], are being explored for solutions.

Examples of other issues being tackled include the oracle problem due to the blockchain's inability to push or pull data from the external environment, privacy of recorded transactions, and compatibility issues across smart contracts of different blockchains. Comprehensive literature review papers on these other issues have been published with examples being [5]-[13].

In developing smart contracts, blockchains have complex constraints for applications, one being the maintenance of smart contracts that is challenged by the fact that smart contracts are also stored on the blockchain that is immutable. These constraints mean that it is even more important to use software and security engineering best practices in new approaches and methods for

¹ Corresponding author

introducing efficiencies as also recommended in [9]. We first review research on development of smart contracts that are closest to our work and then describe our goal and outline further sections.

A. Smart Contract Development

Smart contracts and their development and maintenance received much attention by the research and development communities as can be seen by literature surveys in [5]-[13]. Research on smart contracts has concentrated on (a) the analysis and verification of contracts that are written already and (b) on approaches to create smart contracts from some higher-level model. In [7], research on smart contracts is classified into categories of: (i) Smart contract testing, (ii) Smart contract code analysis, (iii) Smart contract code metrics, (iv) Smart contract security, v) application performance measurements, and (vi) Blockchain applications. Since blockchains are immutable and the smart contracts are stored on the blockchain, fixing blockchain bugs in smart contracts easily is an open issue. Hence, we observe a lot of research in the categories listed above that concentrate on analyzing existing smart contracts for security, vulnerabilities, correctness, and presence of errors.

Khan et al. [6] survey and classify research on blockchain smart contracts into two broad categories of *improvement* and *usage*. The *usage* category reviews how blockchains are used by applications of various types, while the improvement category includes research and tools to improve smart contracts either when writing them or once they exist. Approaches listed in the survey on writing of smart contracts rely on defining a programming language, for instance domain specific or contract-oriented, that facilitates creation of smart contract with the desirable properties. These approaches, although successful in that they lead to smart contract programs with formally proven safety properties, have not been adopted in practice as they exhibit perceived “complexity” by developers who ultimately determine how well-accepted a programming language is.

An extensive literature survey of close to 200 papers on formal specification and verification of smart contracts is provided in [13], in which formalism taxonomy was used to classify work into contract-level and program-level categories, depending on the level of abstraction used in modeling and analyses. Further classification was performed depending on the type of abstraction used for analysis. Contract-level category was further classified into methods using (i) process algebras, (ii) state transition systems, and (iii) set-based methods. Furthermore, the program-level category was classified into (i) abstract syntax trees, (ii) byte code, (iii) control-flow graph, and (iv) program traces.

Literature survey in [11] reviewed existing Model Driven Engineering (MDE) approaches to creation of smart contracts. One of the classifications offered was based on modelling techniques that included UML, BPMN, ERD, DEMO, ArchiMate, and EMF. Our work is closest to research that uses BPMN for business process modeling, which is then transformed into a smart contract. Early work included (i) [14], in which authors describe how UML, BPMN, and ER models could be used to model blockchain-based applications but offered no concrete proposals and (ii) [15] in which authors describe challenges arising when analyzing the blockchain technology using BPM lifecycle as a framework. They also identify a number of issues beyond the BPM lifecycle management. Their analyses led them to present seven major research directions that will emerge in future.

Weber et al. [16] propose architecture for creating smart contracts from a distributed application specified as a BPMN model. The transformation process has two phases, design and execution. In the design phase, the PBMN model is analysed and transformed into a distributed application that uses a blockchain smart contract. There are two components to the system, an off-chain component that accepts input from the external application and transforms it into the calls to the methods of a smart contract, and the on-chain component that includes the smart contract methods and a run-time monitor/mediator that manages the choreography of task activities represented by BPMN elements. From this work, there were two significant follow up projects, Caterpillar [17] and Lorikeet [18].

Caterpillar has been further augmented with the ability to bind actors to roles dynamically using a proposed policy specification language based on Petri net semantics [19], [20]. The proposal has been incorporated into the Caterpillar project software and the approach was evaluated with positive results. The policy specification is transformed into methods of smart contracts used for enforcement. Ciccio et al. [21] also support model-driven engineering using BPMN for generation of the methods of smart contracts and for demonstration of feasibility they use the Caterpillar [17] and Lorikeet [18] projects.

Loukil et al. [22] also start with a BPMN model, but one of their requirements is to address the issue of a smart contract immutability and hence requiring creation and deployment of a new contract any time new updates are required. Thus, instead of generating a compiled version of a smart contract from a BPMN model, they advocate creation and deployment of one generic smart contract that invokes predefined functions. They propose a system architecture that has three layers: Conceptual, Data, and Flow layer. The conceptual layer performs a set of activities that invoke functions of the Data layer.

Lu et al. [23] used business process modeling to support fungible and non-fungible asset management. In addition to BPMN modeling, their architecture also includes asset registry modeling for each fungible and non-fungible asset. For evaluation they use Lorikeet for BPMN modeling of selected use cases from government and industry, wherein the use cases include fungible/non-fungible asset registration, escrow for conditional payment, and asset swap.

In a research report [24], the authors take a different approach in transforming a BPMN model into a smart contract. Instead of starting with a BPMN collaboration represented by a BPMN diagram, as does most of other research on transforming BPMN

models into smart contracts, they start with a BPMN choreography, which describes conversations amongst the participants but not the details on messages exchanged during the conversation. Thus, BPMN conversations within a choreography are not deterministic as there may be many collaborations within a conversation that satisfy the conversation specification and the research work concentrate on optimizing each conversation and their synthesis.

We also mention work in [25] and [26] in which a smart contract is modeled as finite state machines through a graphical editor. In the initial phase, an FSM representing the smart contract is created and transformed into a set of methods of a smart contract. After this preliminary phase, each method of the smart contract is examined using their FSolidM tool to plug the security holes. A contract method is then amended with the security code after which the developer is unable to amend it.

B. Goal and Approach

The main goal of our research is to transform a business application, expressed as a BPMN model, into a set of methods of a smart contract(s). The main distinction of our approach to other work that transforms BPMN to a smart contract, is that we use DE-HSM modeling, in which concurrency is modeled using Discrete Event (DE) modeling and functionality is expressed using Hierarchical State Machine (HSM) modeling. As HSM is a form of Finite State Machine (FSM), one of the basic modeling techniques in computer science, there is rich theory and many tools that have been developed to reason about FSMs.

However, most importantly, using DE-HSM modeling enabled us to create methods that identify patterns, DE-HSM sub-models, that are suitable to be processed on sidechains. We exploit this property to facilitate:

- Cost reduction by allowing the modeller to select patterns, identified DE-HSM sub-models, to be processed on sidechains that are cheaper than the mainchain.
- To achieve privacy by processing specific collaboration activities on a sidechain where data is visible only to the collaboration participants as opposed to all actors accessing the smart contract.

C. Related Work and Contributions

Closest to our work is research on transforming a BPMN model into smart-contract methods with minimal intervention from the developer [16]-[23] that were already discussed in a previous subsection. They transform a BPMN model into a set of tasks that are executed under the control of a monitor. The monitor and tasks are a part of the system deployed in a form of tasks that are executed under the control of a monitor/mediator that is a part of the smart contract deployed on a blockchain. The off-chain part of the system transforms application's input into calls to a smart contract. Input is parsed to determine application's input and choreography is used to determine which task is to be executed. Incorrect sequences of input form "non-conformant" input streams that need to be identified by the system, while correct sequences should result in execution of correct tasks.

We depart from the related work in several major aspects that also form our **contributions**:

1. We use DE-HSM multi-modal modeling that transforms a BPMN model into DE-HSM model in which sub-models are FSMs and DE modeling is used to model time.
2. We support automatic transformation of selected DE-FSM sub-models into the methods of a smart contract. Choreography of collaboration tasks is represented by a DE-HSM model consisting of DE-FSM sub-models.
3. Furthermore, the DE-HSM model representation of the BPMN model into individual DE-FSM sub-models is such that each individual model is an independent component, also referred to as a pattern, with a desirable property that once the pattern execution starts, it is localized until execution leaves that pattern. This property is exploited in facilitating off-chain processing of such patterns and to support privacy.
4. Modeler is provided information on the patterns/sub-models and selects from them those that should be processed off-chain. The TABS system transforms the DE-HSM model into smart contracts, one deployed and executed on the mainchain and the other smart contract deployed and executed on a sidechain (we currently support off-chain processing on sidechains only). The mainchain contract invokes the methods of the sidechain contract that implements the patterns that were selected by the developer to be executed on a sidechain. The mainchain may be either one of Ethereum (private or public) and Hyperledger fabric, while sidechain may be one of Quorum (Ethereum-EVM sidechain) or Hyperledger fabric channel.
5. The use of DE-HSM modeling led to automated support for sidechain processing that is used (i) to reduce the cost of processing by moving computation from the mainchain to a cheaper sidechain and (ii) to achieve privacy in collaboration, wherein data required for use in a pattern that is processed on a sidechain is private and hence is inaccessible to actors who do not participate in collaborating on that pattern.

D. Outline

In Section II, we provide background. Section III overviews our architecture and also presents assumptions and BPMN modeling features that our system currently supports. Section IV describes (i) transformation of a BPMN model into a DE-HSM model and then (ii) creation of DE-FSM sub-models. Section V describes how the DE-HSM model is transformed into the methods of a smart contract(s) and how the modeler is provided with the choice of deploying sub-models for processing off-chain to: (i) decrease the cost of processing by performing work off-chain where processing is less costly and/or (ii) to provide privacy. Section 6 describes our Proof-of-concept system, TABS tool, and its evaluation. The last section offers concluding remarks.

II. BACKGROUND

In this section, we first briefly review the trade finance applications and their properties. We then describe DE-FSM multi-modal modeling that combines DE modeling for concurrency with FSM modeling. We conclude with an overview of BPMN elements.

A. Trade Finance Applications

Blockchains and their smart contracts are not suitable for all types of applications. Smart contracts are not suitable for applications that exhibit heavy computational requirements as execution of smart contract transactions requires validation involving all or a number of the blockchain nodes. As their name implies, smart contracts are particularly suitable for applications that involve collaboration of different actors, possibly of different organization, with varying requirements on information availability, security, and privacy, amongst others. And trade applications involve activities that exhibit those properties and thus should be suitable for blockchain applications - as is confirmed by a much higher adoption of blockchains in trade, which also includes finance in trade and hence also Decentralized finance (DeFi). Thus, trade and finance applications are our target area, which is a broad area involving third parties for the purposes of efficient processing of transactions by participants, such as banks, customs, shippers, and insurance companies.

B. FSMs, Hierarchical State Machines, and Nested Multi-modal Modeling

FSM modeling has been used frequently in the design and implementation of software and adaptations of FSM modeling techniques were proposed to expand the applicability of FSMs to modeling applications. For instance, a guard along an FSM transition is used to specify a Boolean condition on the state's variables that must be true for the transition to take place. In the late 80's, FSMs were extended to address the issues of reuse of patterns with the concept of hierarchy, leading to Hierarchical State machines (HSMs) that may contain states that are themselves other FSMs. Harel [27] showed that FSMs can be combined hierarchically: A single hierarchical state at one level can be considered to be in several states concurrently as represented by an FSM(s) in a lower level of the hierarchy, and FSMs may also be combined leading to concurrent FSMs.

An HSM can be defined using induction as follows (due to [28] and also described in [27], [29], and others): In the base case, an FSM is a hierarchical machine. Suppose that M is a set of HSMs. If F is an FSM with a set of states, S , and there is a mapping function $f: S \rightarrow M$, then the triple (F, M, f) is an HSM. Each state, $s \in S$, that represents an HSM is replaced by its mapping $(f(s))$. HSMs recognize the same language as its corresponding flattened FSM. However, HSMs do not increase expressiveness of FSMs, only succinctness in representing them.

Girault et al. [28] describe how HSM modeling can be combined with concurrency semantics of a number of concurrency models, including communicating sequential processes [30] and discrete events [31]. They describe how an HSM model can represent a module of a system under a concurrency model that is applicable only if the system is in that state. This enables representation of a subsystem using a particular concurrency model that may be nested within a hierarchical state of a higher-level FSM. This may be used in multi-modal modeling, in which different (hierarchical) states may be combined with different concurrency models that are best suitable for modelling of concurrent activities for that particular state. We exploit the concept of multi-modal modeling to allow the designer to model concurrent, but independent activities, by concurrent FSMs at the lower level of hierarchy, in order to avoid the state explosion that may arise in such situations [28]. In [32], we showed that a multi-modal model that combines Discrete Event (DE) modeling with FSM modeling may be used to model trade finance applications – such a combination of models is referred to as DE-HSM multi-modal modeling. We also showed that it is possible to automatically transform a DE-HSM model of such an application into methods of a smart contract deployed on a blockchain.

A DE-HSM has external inputs, and it produces outputs. The model represents how external inputs form inputs to sub-models and how those sub-models are interconnected to produce the final output. However, if the sub-model's interconnection is such that there are no loops, then the model can be viewed as a zero-delay model [29] in which the individual DE queues may be combined into one DE queue.

Feedback loops in a DE-HSM arise due to feedback loops in the BPMN model from which the DE-HSM model is derived. None of the approaches, to producing smart contract from a BPMN model that were discussed in the introductory section, addressed the issues of the feedback loops in a BPMN model. Feedback loops cause various difficulties, such as deterministic semantics, and will be discussed further in a later section.

In modeling, the term DE-FSM modeling refers to a model for which it is known that they do not contain hierarchical states.

C. BPMN Overview

BPMN is viewed as a de-facto standard for describing business processes. It was developed by the OMG organization with the objective of BPMN to be understandable by all business users, from business analysts, through technical developers implementing the processes, to people managing those processes. That it has been adopted in practice is demonstrated by many software platforms available that provide for modeling of business applications with the objective to automatically create an executable application from the BPMN model.

For instance, Oracle Corporation uses the Oracle Business Process Analysis Suite to model business processes using BPMN and transform a BPMN diagram into a blueprint for an executable Business Process Execution Language (BPEL) processes [33], wherein the blueprint represents the logic of the application in terms of concurrent processes and their interactions, while details of individual tasks are supplied by implementors. Another example is Camunda software platform that is also used to develop a BPMN model that is then transformed automatically into a Java application. Our approach is similar, but instead of targeting BPEL or java as the target for creating executable applications, we target smart contracts executing on blockchains.

BPMN standard categorizes the BPMN elements into five basic categories: (i) *Flow objects*, (ii) *Data*, (iii) *Connecting objects*, (iv) *Swimlanes*, and (v) *Artifacts*.

1) Flow Objects

They include Events, Activities, and Gateways.

- *Events*: There are three types of events: *Start*, *Intermediate* and *End*. Start and End events are special in that they denote the start and an end of an activity or a business process. Intermediate events include many different types with examples that include message timer, timer, conditional, escalation, and multiple parallel. Each type has a special symbol with an icon having special marking to indicate properties, such as throwing or catching, interrupting and non-interrupting, and boundary. Some of the throwing events are connected to catching events with sequence flow to show the flow of control.
- *Activities*: As the name implies, activities represent work performed in a business process flow, activities that include *Tasks*, *Sub-processes*, and *Call Activities*. Call activities allow re-use of Tasks and Sub-processes and thus enable their nesting. An activity may be performed once or many times, in which case it is a *looped* activity. A looped activity is a specified activity that is intended to be performed on each of a set of objects that may be viewed as input to the looped activity. The looped activity may be either *Standard*, wherein the specified activity is executed sequential once for each object in the input set of objects. Or it may be a *Multi-Instance* activity, in which case the activity is executed concurrently on each object in the set of objects.
 - *Task* is an activity that is independent of other activities or BPMN elements.
 - *Sub-process* is a compound activity that can be represented by other elements of which scope is limited to the sub-process itself. A sub-process may be defined further as a *transaction*, in which case the sub-process must be supported by a special protocol that ensures that all of the transaction activities have completed successfully and, it that is not possible, that all activities completed partially by a sub-process are “undone” by performing compensating activities. Compensating activities are shown using a *Compensating association*.
- *Gateways*: They are used to represent the flow of a business process. A gateway may (i) have one or more incoming/converging flows to form a join/merge gateway, (ii) have one or more outgoing/diverging sequence flows, in which case it is a split/fork gateway. If a gateway has multiple incoming and multiple outgoing sequence flows, then it is both a converging and a diverging gateway.
 - A *diverging exclusive* gateway, which may have conditions specified on the outgoing paths and permit a default outgoing path, allows only one outgoing execution at a time.
 - *Inclusive diverging* gateway allows many outgoing sequence flows to be active. If an outgoing flow has an associated condition, then that condition must evaluate to be true at run-time in order for a flow to proceed along a path.
 - A *parallel converging gateway* does not have conditions on its path: Each incoming sequence must receive a token (an incoming flow must arrive on a path) before triggering the outflow. Once an outflow is triggered, all outgoing paths are activated and proceed in parallel if the gateway is also diverging.
 - An *inclusive converging gateway* passes token only if a token has arrived on each of the enabled incoming paths; otherwise, there is a wait until the tokens do arrive.

2) Data

Included are *Data Objects* that provide information about what data activities use or produce. A *Data Object* is used to represent either an object or a set of objects. *Data Input* provides input information to processes, while *Data Output* is used to receive information from processes. A *Data Store* use is self-explanatory.

3) Connecting Objects

Connecting objects are used to indicate the sequence of execution. Connecting objects can be (i) *Sequence Flows*, (ii) *Message flow*, (iii) *Associations*, and (iv) *Data Associations*.

- *Sequence flows* show the normal flow. In addition, there are specialized types of sequence flows, namely *Conditional flow* that has a condition that is evaluated at run-time to determine if the flow may proceed and *Default* and *Exception flows* that are self-explanatory.
- *Message flow* shows the flow of messages.
- *Association*, such as a *Text association*, is used to link information and Artifacts with BPMN graphical elements

4) Swimlanes

A *swimlane* represents a participant in a business process. There are two types of swimlanes: *Lanes* and *Pools*. A pool with lanes represents an organization while lanes within a pool represent individual participants within that organization. Only message flows can cross pools - no other BPMN elements can do so.

5) Artifacts

They are used as a form of documentation to provide additional information about a process. The two standardized Artifacts are *Group* and *Text Annotation*, but modelers are free to add as many Artifacts as necessary. *Group* provides for a graphical representation of a collection of elements. *Text Annotation* is used to provide additional textual information to an BPMN diagram element.

In the version 2.0.2, OMG introduced choreography in BPMN to represent the way business participants coordinate their interactions through conversations, which is a higher-level of abstraction concept than the core BPMN elements. A choreography hides many details that are required for elaboration of a BPMN model into an executable application regardless of whether it is blockchain based or not. In a BPM model, a choreography focuses on showing message exchanges amongst the participants, however, without providing the details on the messages exchanged or their content. In a BPMN choreography, instead of showing detailed message exchange, commands of higher-level abstraction are used to represent the exchange of messages using a single request-reply type element.

III. SYSTEM ARCHITECTURE

Although we discuss our assumptions throughout the paper, we enumerate them collectively in this section. They include assumptions on the BPMN model and which BPMN elements we support. We then provide a high-level overview of the architecture that consists of two components, one used in the design phase to transform a BPMN model into the methods of a smart contract(s) and one that is used during the run-time phase to monitor and manage the smart contract deployed on a blockchain. Each of the phases is briefly overviewed and then each one is described in more detail in subsequent sections.

A. Assumptions

We first describe assumptions we make regarding a BPMN model and then assumptions made on the system architecture and its implementation.

1) Assumptions on the BPMN models

These assumptions deal primarily with loops, subprocesses, converging gateways, model preprocessing, and which BPMN elements we currently support in the tool.

a) BPMN model with limited support for looping subprocesses and parallel subprocesses.

That loops in BPMN models cause difficulties with BPMN semantics has been recognized and examined by a number of researchers. For instance, Dijkman et al. [34] analyze the semantics of the BPMN model and identify a number of issues arising due to the inexact nature of semantics in the BPMN specifications. And one of the issues identified were difficulties arising due to uncertain semantics of BPMN specification for feedback loops. We do support limited form of looping in BPMN diagrams, wherein the limitation is that a loop is defined only for patterns, that is portions of a BPMN diagram, that can be represented by subprocesses. That is, the BPMN elements of a pattern are connected to the rest of the diagram only by two connecting objects, such that one is incoming, and one is outgoing. In effect, such a portion/pattern of the BPMN diagram can be represented by a sub-process as it too has only one incoming and one outgoing connecting object. In fact, the way we handle

such a pattern with a feedback loop is by transforming into a looping sub-process, but with limitations on the looping conditions, as shall be described under the looping subprocess description below. Interestingly, none of the other approaches to producing smart contract from a BPMN model that we discussed in the introductory section made any mention about how loops in a BPMN model were handled.

We support subprocesses in a straightforward way by replacing each subprocess with its BPMN model of core elements (BPMN diagram of the expanded subprocess) before transforming a BPMN model into its DE-HSM representation. We also support a limited form of *looping* and *parallel subprocesses*.

Parallel subprocess: Our approach is to replace a BPMN subprocess with its elaboration in terms of its activities expressed using the individual BPMN elements when the subprocess is expanded. To support parallel streams for a parallel execution, our approach needs to know the maximum number of parallel activities in a process as we need to create the required number of streams, one for each parallel activity, before the smart contract deployment. Before transforming the DE-HSM model into a smart contract, the modeller needs to specify the maximum number of parallel streams for a parallel subprocess. That information is then used to create the number of maximum parallel streams, which exist in the smart contract whether they are used or not. Thus, we do not fully support the BPMN semantics in that there may be an arbitrary number of separate parallel streams that are used to process a collection input into a subprocess, wherein the collection has an arbitrary number of elements that results in an arbitrary number of parallel streams, each one to process one element of the collection. However, this issue arises generally with handling looping mechanisms in deterministic systems, such as systems expressed in FSM or HSM modelling.

Looping subprocess: We do support a limited form of a looping subprocess, wherein the limitation is that the maximum number of iterations is known before the start of the execution of a looping subprocess. We model a subprocess without its looping property and we accommodate the looping after modeling, before the creation of a smart contract in a way that is similar to the usual *for-loop* programming construct. Transformation prepares inserts script for a looping variable just before entering the looping sub-process, with the looping variable initialized based on the size of the input collection. Furthermore, the translator inserts script at the end of the looping subprocess to increment the looping variable and check whether to perform the loop again.

b) Assumptions on converging gateways

Converging gateways may be parallel, in which case for a token to be passed, a token must have arrived on each of the incoming edges of a converging gateway.

However, the converging gateway may also be inclusive, in which case the BPMN semantics state that when a token arrives on an incoming edge to an inclusive converging gateway, the token can be passed only if a token has arrived on each of the incoming pathway that is enabled; otherwise there is a wait for the remaining tokens arriving on such enabled pathways. However, *we make a simplification for an inclusive converging gateway* in that we simply pass the token - we check the other pathways neither for enablement nor for a token arrival.

c) Assumptions on BPMN model preprocessing

Once a BPMN model is created and expressed using its standard XML representation [34], we assume that the model is transformed into an equivalent BPMN model that is well-formed, wherein the well-formed BPMN model is defined as in [34]: A BPMN model is well-formed in it satisfies the following conditions: (i) there is only one start event; (ii) there is only one end event; (iv) fork/split decision gateways have one incoming flow and more than one outgoing flow (v) join/merge gateways have one outgoing flow and more than one incoming flow; and (vi) there are no data-based splits/forks or joins/merges – they are expressed using equivalent gateways constructs. The above conditions are not restrictive as any BPMN model can be transformed into an equivalent well-formed BPMN model as described in [34].

The proposed TABS tool, described at the end of this paper, allows users to compose BPMN diagram directly on the canvas by invoking API provided by Camunda [35]. The model is saved by the tool in the BPMN XML representation [36] and then is transformed into a DE-HSM model by our TABS tool. The BPMN model may contain participants (Pool and Lane), activities (tasks, subprocesses and transaction), gateways, and events (default, start, end, message, and timer) [37]. Fig. 1 shows the symbol that our TABS tool supports, albeit as noted in III.A.1(a), we impose limits on looping and parallel subprocesses and arbitrary loops in BPMN diagrams.

2) Other assumptions

a) No support for long-term trade transactions

BPMN has a concept of a transaction defined on a subprocess and a blockchain also supports transactions. That is, a subprocess can be defined in BPMN as an atomic transaction in that the whole subprocess needs to be either completed entirely, or none of its effects, such as updates to the state variables, be recorded on the blockchain. Any activity that was performed by a subprocess that was subsequently aborted must be “undone” by compensating activities. Blockchains also have transactions of which semantics are defined in terms of blocks of transactions. Furthermore, trade applications also have a concept of a

transaction, wherein transactions may be defined for a group of activities that may span long periods of time and may apply to data that is a subset of data processed by a trade application. The problem is that these three concepts of a transaction, one being for the trade vertical, one for BPMN, and one for blockchains, are mismatched as they do not align, which is akin to the *impedance mismatch* between the relational and object-oriented DBs [38]. We do not address this mismatch between the BPMN, blockchain and trade transactions, in this work, but we do plan on doing it in our near-future work.

Supported BPMN Symbols								
Participants								
Gateways		 Exclusive			 Inclusive		 Parallel	
Activities		 Task			 Subprocess			
Events								
	Start			Intermediate			End	
Type	Normal	Event Sub process	Event Sub process non-interrupt	Catch	Boundary	Boundary non-interrupt	Throw	
None								
Message								
Timer								
Conditional								
Signal								
Terminal								

Fig. 1. BPMN symbols supported by TABS

b) Off-chain storage

It is well known that storing objects or data on a blockchain is expensive, particularly for a public blockchain, such as Ethereum. It is thus a standard practice to store large objects/data off-chain and store on the mainchain only the hash-code of the data stored off-chain. Furthermore, the hash-code of an object is used any time the object is retrieved by a smart contract to ensure the immutability property of an object stored off-chain. Thus, we also assume that any objects are stored off-chain. An exception to that is any object used in support of the monitor of the architecture shown in Fig. 2. We facilitate off-chain storage by using IPFS. IPFS is a distributed system for storing and accessing files, websites, applications, and data [39]. It is resilient and has a decentralized control. Due to its resiliency, decentralized control, and content addressing using a hash-code, it is particularly suitable as an off-chain storage for blockchains. IPFS storage ism, content addressable using the hash-code of an object.

c) BPMN Task element

The BPMN task element represents a self-contained task. However, our system does not provide the script for the task element, the developer does. Our system does prepare the method skeleton, including the method's signature, and the method's invocation, while the developer is asked to supply the code for each task before deployment. This approach is standard in creating applications from a BPMN model. For instance, Oracle Corporation uses the same approach in its Oracle Business Process Analysis Suite when transforming a BPMN diagram into a blueprint for executable Business Process Execution

Language (BPEL) processes [40] and a similar statement also applies to the Camunda platform that also asks the modeler to provide the script for each of the BPMN model's tasks.

B. Architecture Overview

Architecture of the system is shown in Fig. 2, which shows two major phases: Design and Deployment/Installation phase, and Execution/run-time phase. In the first phase, the BPMN diagram is first transformed into the DE-HSM model and then into the methods of a smart contract. However, besides the creation of the smart contract methods with its supporting run-time management information, the transformation also prepares API methods to be used by a **Distributed Application (DApp)** to interact with the methods. Once the smart contract is prepared, it is deployed in the Design and Deployment/Installation phase that deploys the smart contract on the blockchain and also installs the API and associated methods in a network node so that a DApp is able to communicate with the smart contract. The smart contract includes a monitoring module with the run-time management information stored in the data structures prepared during the design phase. They include the DE queue of events and FSM definitions corresponding to the DE-FSM sub-models and FSMs to support concurrent processing within individual DE-FSM sub-models.

After the smart contract deployment and installation of API methods, run-time processing involves the user interacting with the DApp that invokes appropriate API methods in response to user input. This input is examined by the *API-SCmethods* module that translates it into an invocation of an appropriate smart contract method.

A smart contract method input is passed to the Monitor that:

- Accepts inputs from a smart contract method, analyzes them and queues an appropriate event into the DE queue as a response to the input.
- Removes events from the DE queue one at a time, and for each event:
 - Analyzes an event's information and invokes an appropriate FSM while passing it input derived from the event's information formed by input to a smart contract method.
 - The invoked FSM takes the input, examines its state and fires causing a transition while generating output that is returned back to the monitor.
 - Monitor analyzes the output produced by the FSM's firing and it may perform one or more of the following actions:
 - Invoke one of the tasks
 - Queue an event into the DE queue
 - Respond to the smart contract invocation
 - Cause a signal to be delivered to the DApp

C. Design and Deployment/Installation Phase

The design phase consists of the following main steps:

1. Initialization and Transformation of BPMN model into DE-HSM multi-modal model
2. Transformation of the DE-FSM sub-model into a set of methods of a smart contract
3. Deployment and installation: Deploy the smart contract(s) and install the API-SCmethods and the DApp API

As was already indicated in the previous section, our architecture is similar to that of [16]-[18] and, for the sake of comparison, we use the same terminology as in [16], which has also been used in other work. Fig. 3 shows a BPMN model to be transformed by a Translator of the Design phase into a DE-HSM model, which is followed by producing the smart contract methods. Similarly, the run-time phase involves a mediator/monitor that takes external events/triggers, which the DApp delivers to the run-time methods as parameters when calling them. Response is delivered to the DApp via output parameters upon return.

In initialization of the design phase, data structures are prepared for the subsequent design steps that will include in them information that is used during the execution phase. After initialization, transformation from the BPMN model into the DE-HSM model deals with determining which parts of the BPMN model should be represented by which of the DE-FSM sub-models. Each of the DE-FSM sub-models includes a DE model to represent concurrency and an FSM model to represent the functionality. Choreography (not a BPMN term), which orchestrates execution of activities in possibly concurrent processes, is represented by a DE-HSM model that represents the control flow amongst the DE-FSM sub-models, as will be described later. Furthermore, each DE-FSM model may also have internal parallel activities if the BPMN collaboration it represents has parallel or inclusive gateways.

In addition to the smart contract methods and information for the run-time Monitor, the design phase also prepares API methods to be invoked by the DApp. The API methods presented to the DApp are focused on activities amongst actors and thus roughly correspond to the flows of objects between either pools or swimlanes. As a consequence, when input is received via a message from one actor to another, receiving a message will result in a call to the API with input being the content of the received message. The Design phase prepares appropriate information for the API-to-SCmethods translator that assists in

determining which of the smart contract methods should be invoked and which input parameters to the API should be included as input to the smart contract method invocation.

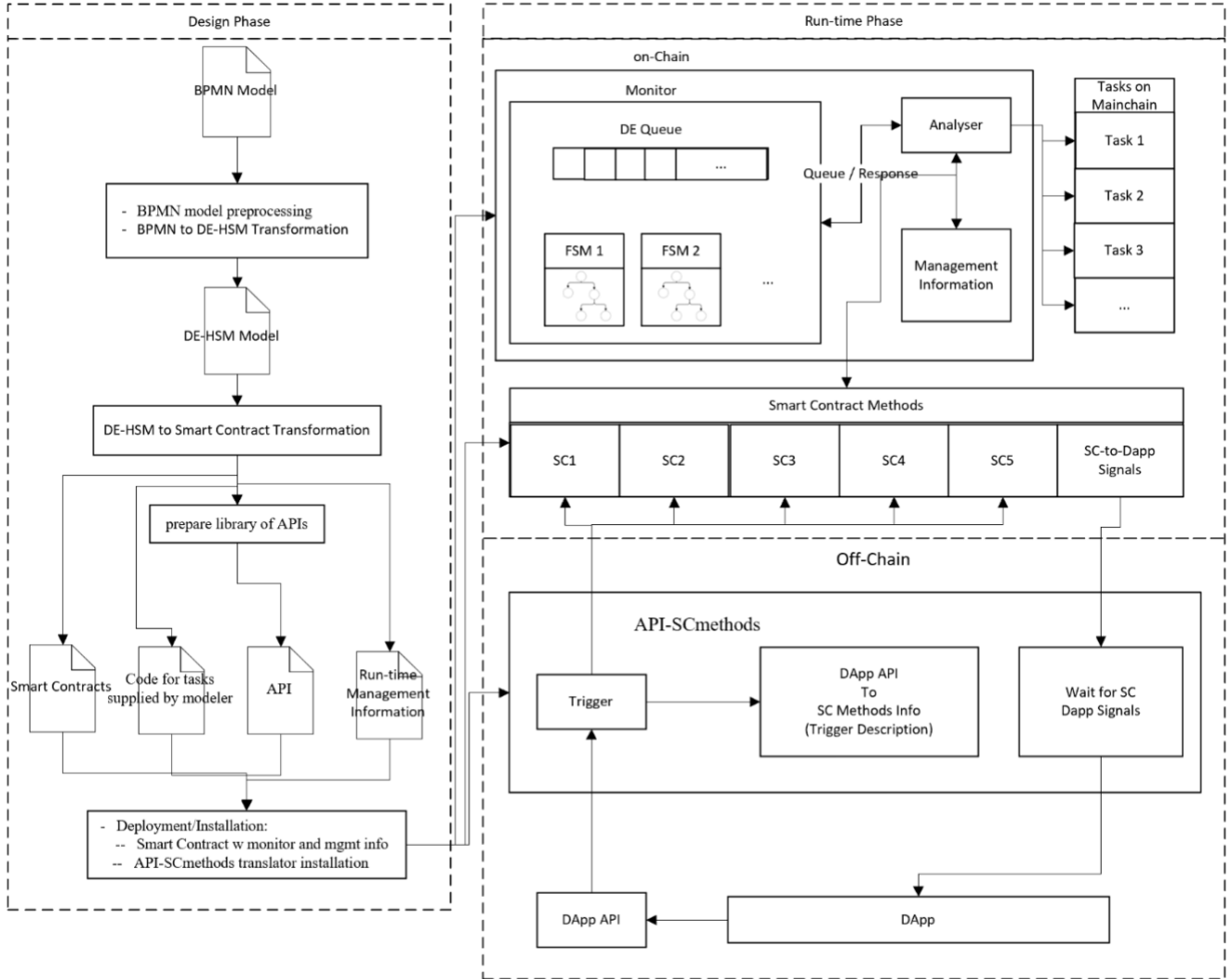


Fig. 2. Architecture Overview

In addition to the smart contract methods and information for the run-time Monitor, the design phase also prepares API methods to be invoked by the DApp. The API methods presented to the DApp are focused on activities amongst actors and thus roughly correspond to the flows of objects between either pools or swimlanes. As a consequence, when input is received via a message from one actor to another, receiving a message will result in a call to the API with input being the content of the received message. The Design phase prepares appropriate information for the API-to-SCmethods translator that assists in determining which of the smart contract methods should be invoked and which input parameters to the API should be included as input to the smart contract method invocation.

The last steps are relatively straight-forwards as they include deployment of the smart contract methods and installation of the *DApp API* and *API-to-SCmethods* modules on network nodes.

D. Execution Phase

Execution phase includes two phases:

1. Initialization
2. Execution

Initialization deals with issues, such as authentication of the actors, also referred to as participants. Once the smart contract is deployed, execution is managed by the Monitor module.

It should be noted that the activities described above need to be performed by any approach that generates smart contract methods from a BPMN model. Consequently, parts of the architecture for the run-time phase are similar to other architectures, such as architecture in [16]-[18]. For that reason, we use the same or similar terminology, including terms of monitor/mediator, and triggers. Fig. 2 shows, for the run-time phase, an off-chain *DApp API* module that provides an API to the DApp. The module also shows an *API-to-SCmethods* component that transforms an API call into a call to a smart contract method. In other research, this component is called “triggers”, but we feel that the *API-to-SCmethods* name is more appropriate than triggers.

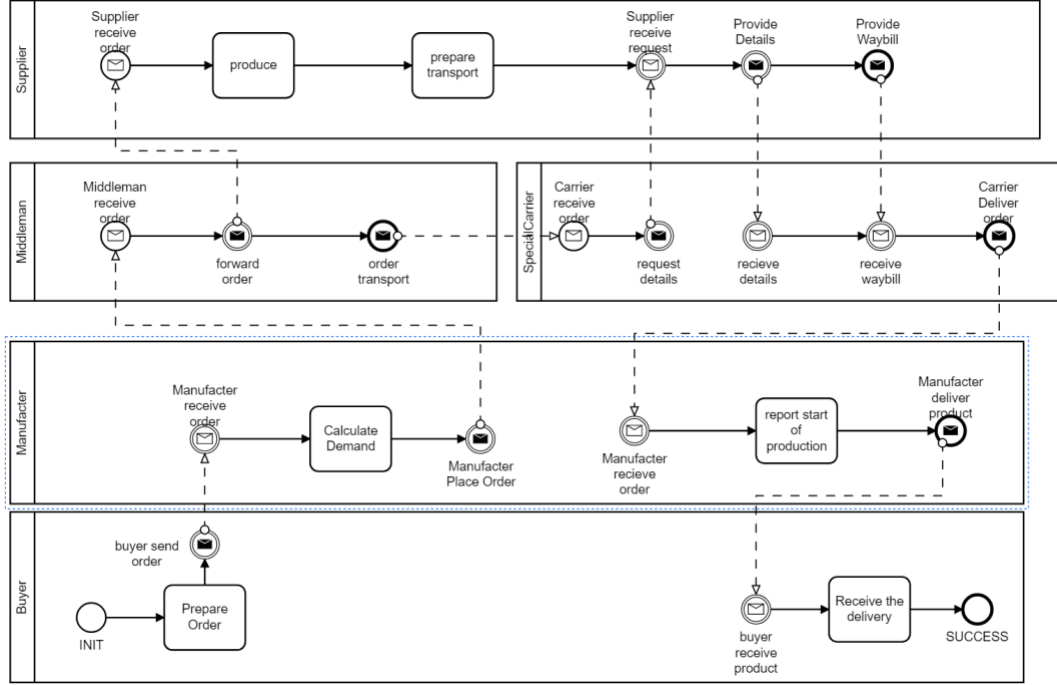


Fig. 3. A BPMN model for supply chain example

The *DApp API* methods correspond to information flow across pools or swimlanes and, when a connecting object crosses a pool or a lane, information about the origin and destination of the connecting object is included in input parameters to the *API-to-SCmethods* component to determine which smart contract method is to be invoked.

A smart contract, at times, needs to inform external actors of certain events arising to which a user should react. For instance, a contract may go through computation along different paths that eventually meet, at which point a user(s) may need to be informed of the results before the computation proceeds. Our systems facilitate this by the API, which is off-chain, having a process that invokes a special API method (in Fig. 2, the process is shown as the *Wait for SC to DApp signals* process) that in turn invokes a smart contract method that waits for a “wakeup” event that may be generated by another smart contract method. When a smart contract needs to signal the external DApp, it raises a “wakeup” event that releases the method waiting for a wakeup event and it can thus return to the DApp information about an event that has occurred within execution of the smart contract.

When a DApp makes a call to the *DApp API*, the call information with its parameters is provided to the *API-to-SCmethods* module that determines which smart contract method is to be invoked, and then it invokes the smart contract method and waits for its response.

The smart contract method provides its information to the *Monitor* that uses its *Analyzer* sub-module to examine the input parameters and use them to queue an appropriate event into the DE queue. When that event reaches the head of the queue, it is dequeued and passed to the appropriate FSM for firing. The FSM receives its input, checks the current state, and fires, that is, it makes a transition while generating output. The transition’s output indicates if a task or a subprocess, associated with the new FSM state, needs to be performed. For instance, if the transition was to a state that represents a task *calculate demand*, then a task created for that purpose is invoked. As a part of the FSM transition, the smart contract analyzes the returned output from a transition and consults information on tasks and subprocesses and determines which of the following should be performed: (i) schedule another event by queueing it into the DE queue; (ii) prepare output parameters and return it to the smart contract

method that invoked it; (iii) produce output and cause a DAapp signal to be generated; (iv) invoke another smart contract method.

E. Example

Consider the BPMN diagram in Fig. 3 and assume that processing has progressed successfully and that (i) the manufacturer has sent a message to the middleman with an order for parts; (ii) the message with the order has just been received by the DApp, and that the DApp just invoked the API with information about the received message. The API method takes the information from the API method and packages it as input to the *API-to-SCmethods* component together with information on the source and destination of the message flow-object. The component consults its information, invokes an appropriate smart contract method while passing it input parameters and then waits for the response from the smart contract method.

The smart contract method passes its information to the *Analyzer* component of the *Monitor*. The Analyzer checks its management information and transforms the smart contract input information into an event, which includes parameters, that is stored into the DE queue. The component then removes elements from the DE queue, processes them, and repeats until the queue is empty. In this example case, there is only one element that was dequeued from the DE queue, an element representing the reception of the message containing the order.

When the event, which represents the reception of an order, is dequeued from the DE queue, it is processed by the *Analyzer* by: (i) determining which of the FSMs is to be fired to process the dequeued event; (ii) preparing the input parameters for the FSM; and then firing the FMS, i.e., causing the FSM's transition. The FSM takes the input, the current FMS state, and determines the transition while producing output that contains the message content and also information about the new state that corresponds to a task *ProcessOrder*. Since that state has been reached and it represents the *ProcessOrder* task, the corresponding task is invoked that produces output from the transition that includes the processed order. As the output from the task, the processed order, is passed to the next element, it is returned to the *Analyzer* as transition's output. *Analyzer* then examines the output from the transition and checks the new state, which is sending a message to the Manufacturer and is still a part of the same FSM. Consequently, *Analyzer* creates a new event with input containing the processed order and queues it into the event queue – and the process repeats.

IV. TRANSFORMING A BPMN MODEL INTO A DE-HSM MULTI-MODAL MODEL AND ITS DE-FSM SUB-MODELS

To better understand the transformation process, we briefly review the key properties of the DE-HSM modeling. As our system transforms a BPMN model into a DE-HSM model, we need to identify the DE-HSM sub-models and their interconnection. However, before we proceed, we need to be aware of the model semantics and its implications on modeling. Furthermore, we also explain some of the more subtle points on how certain BPMN elements are represented in the transformation process. Only then we describe the transformation process itself.

A. DE-HSM Modeling Semantics and Loops

We use DE modeling for concurrency combined with HSM modeling for functionality, wherein the modeling is applied hierarchically. As an example, Fig. 4 shows a system that contains two sub-models.

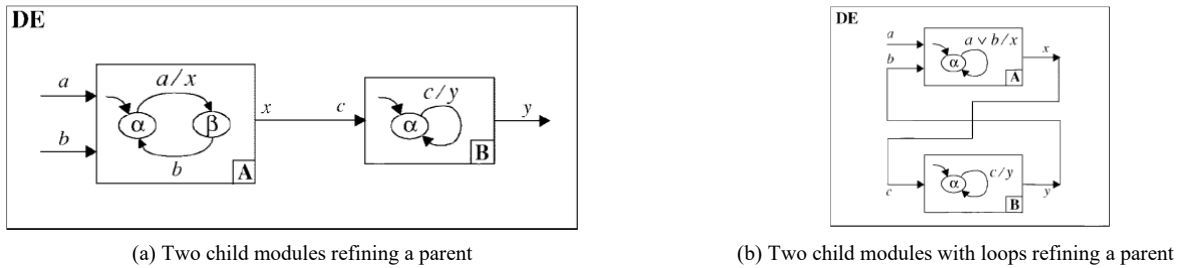


Fig. 4. Two examples of DE-HSM models (Adopted from [28])

Fig. 4(a) shows a DE-FSM master model containing two DE-FSM sub-models, A and B, and their interconnection. Each of the master and the two sub-models has its own queue of events for DE modeling and an FSM for functionality modeling. It should be noted that we referred to the model as a DE-FSM model as the subsystems have FSMs and not HSMs.

In semantics for DE-HSM models, adopted from [28], an FSM system appears to the DE system as a zero-delay participant/actor. Under the assumption of zero-delay semantics, a system with sub-models is such that the FSM of a child subsystem is the first to fire, that is it performs the transition on inputs and produces output that is timestamped with the same

timestamp as input (hence zero-delay), and only then does the parent system react and makes a transition that causes its output. Furthermore, without loops, we can use one global queue of events, instead of each subsystem having a separate queue of events [28].

However, loops cause difficulties under the assumed zero-delay semantics. Consider Fig. 4(b) and the two subsystems A and B. There is a loop because output from A forms input to B, and there is another loop in which B's output is input to A. Such loops cause difficulties, and we cannot apply zero-delay semantics in such a case, as execution may never terminate. Consequently, representing a BPMN model using DE-HSM modeling, we wish to avoid such feedback loops due to the semantic difficulties described above. In the section III.A, we already assumed that a BPMN model does not include any loop that cannot be represented by a looping subprocess. Furthermore, the mechanism for a looping subprocess is implemented as a part of the transformation from the DE-FSM model into the methods of a smart contract that occurs after the DE-HSM model is created from a BPMN model.

B. Initialization (in Design Phase)

In the initialization phase, the translator first transform a given BPMN model into an equivalent model that is well-formed as described in Section III-A.

In general, a BPMN specification does not require the modeler to provide description on all information, which is associated with the BPMN model, that is required for execution. However, since our objective is to produce executable smart contracts, the TABS system must obtain such information from the modeller. Thus, in the initialization phase, the software interacts with the modeler to ensure that transformation has information about all data flowing along the edges of a BPMN graph to ensure that transformation has all necessary information about guards on gateways and about the content of messages. During initialization, we ask the modeler to annotate the BPMN graph accordingly using data associations. The issue is that BPMN specification does not insist on the modeler to provide such information and hence all approaches that strive to create an executable smart contract from a BPMN model need to seek information from the modeler if it is missing in a BPMN model [35].

Once the modeler supplies information on the data flowing along the connecting objects, the system asks the modeler to provide the code to be executed by each individual task. Each task has a single input and a single output, wherein each may be a complex object. The tasks are incorporated by the system into the smart contract methods so that they can be executed according to the control flow as specified by the BPMN model.

C. Representing BPMN elements for DE-HSM Modeling

Transformation of a BPMN model into a DE-HSM model is based on a graph representation of a BPMN model given its BPMN diagram description using XML. For the DE-HSM modeling and the transformation process, we use a *Directed Acyclic Graph (DAG)* representation of the BPMN diagram. In the DAG representation, each BPMN flow element/object is represented using a graph vertex, while connecting objects, that is sequence and message flows, are represented by edges in which the flow control is indicated by the edge direction.

For most of the BPMN flow object elements, the flow-control graph representation is straight-forward in that it can be represented by a graph vertex, with one or more input flows and with one or more output flows. For instance, the task and subprocess BPMN elements belong to the category of *activities* of BPMN symbols are represented as graph nodes, each with one incoming and one outgoing flow object. A gateway can also be represented by a graph node with one or more incoming or outgoing edges. *Events*, which also belong to the category of activities, are not as obvious and shall be discussed here in more detail in terms of how they are represented in a DAG used for DE-HSM modeling.

There are three types of events: *start*, *intermediate*, and *end*. The *Start* and *End* events are special in that they occur at the very beginning and the very end of a process, respectively – hence they are specialized cases of the flow of control. Events have a *Dimension* attribute that describes the type of an event, which may be one of: *message*, *timer*, *cancel*, *compensation*, *conditional*, *link*, *signal*, *terminate*, *multiple* and *parallel*. Out of this list, we currently support the following events: message, timer, conditional, signal as shown in Fig. 1. We are in the process of incorporating the support for the rest of the BPMN elements. We first discuss how *intermediate* events are represented and then discuss representation of the *begin* and *end* events.

Intermediate events that are not boundary have explicit connections, represented by connecting objects between the corresponding throwing and catching events. Thus, they can be represented by vertices connected by edges that represent the connecting objects. A boundary event does not have an explicit connection via a connecting object to its counterpart, such as which signal in a task/subprocess is handled by which of the boundary catch events. An explicit connection between the rise of an event and its consequence, however, needs to be established, which is done at the conclusion of the modeling process when the modeler is asked by the transformation process to provide details on the flow of inputs and outputs along the edges of the graph. For each boundary catch event, the modeler is asked to identify, by using labels, which of the signal event caused the catch event by matching the labels of the throw event firing and its catching it as a boundary event. Thus, each event does have

at least one incoming and one outgoing edge. A boundary catch event, such as catching a signal event, is represented by a graph node that is connected to its corresponding throw signal event, within the activity to which it is attached (as a boundary event), by an incoming edge from the vertex representing throwing the corresponding event. If the boundary activity is also interrupting, the state of execution for the interrupted activity is ended. However, if it is not interrupting, in essence two concurrent activities will proceed, one to continue with the execution of the activity with the boundary condition and one due to the catching of the boundary non-interrupting event and processing it.

A boundary *begin event* is a throwing event. If a boundary *begin event* is on a boundary of an activity, i.e., a subprocess or a task, that is a graph node B, the boundary event is represented on the DAG representation of the BPMN model by inserting an inclusive gateway on the incoming edge to the node B as is identified via label numbers between the boundary start event and the label of its corresponding catch event as provided by the modeler. In other words, boundary throwing events are fired/thrown at the beginning of an activity and, if it is non-interrupting, then two processes proceed concurrently during execution, one for the *boundary start-event* activity and one for the uninterrupted activity represented by the vertex B to which the boundary start event is attached.

A boundary *end event* is a catching event, and it is represented in the DE-HSM model in the usual way by a graph node with the incoming edge that is also an outgoing from the corresponding throwing event as represented by the labeling of events.

D. Transformation of a BPMN Model into a DE-HSM Multi-modal Model

The DE-HSM model, which we are seeking and that represents a BPMN graph, is an interconnection of DE-HSM sub-models, where each sub-model is a child model for the parent model. Furthermore, each of the sub-models may itself be a DE-HSM sub-model, i.e., it may be modeled as an interconnection of its child-models. As described in a previous sub-section, the TABS system represents a BPMN model using a DAG, and then it determines the DE-HSM model as an interconnection of the sub-models, wherein each sub-model is represented without further sub-models: The topic of this section. However, once we do find the sub-models, we also need to describe how to build each sub-model, which is the topic of the next section.

Starting with a BPMN diagram, we first describe how we represent a BPMN model as a DAG and then how we approach defining the problem of finding a DE-HSM model for a given DAG. To find a DE-HSM model, we use the concept of *independent subgraphs* and their variations, which are described next. We then present an algorithm to find a DE-HSM model that is an interconnection of DE-FSM sub-models, i.e., sub-models that do not have hierarchical states.

1) DAG representation of a given BPMN graph

At execution time, a DE-FSM sub-model contains a DE queue of events and an FSM. At execution time, processing withing a sub-model proceeds by dequeuing an event from the DE queue and then using it to form an input that is given to the FSM. Upon input, the FSM fires and produces output. The main property of the DE-FSM sub-model, however, is that it accepts inputs that are fed to an FSM and outputs produced by the FSMs firing produce the output out of the sub-model. Thus, a sub-model consumes one set of inputs and produces one set of outputs.

We represent a BPMN model as a DAG $G = (S, E)$, where S is the set of vertices and E is a set of edges. For a given BPMN diagram, vertices in S represent the BPMN non-connecting objects, while edges represent the connecting objects, that is sequence flows or message flows. We use a DAG $G^\wedge = (S^\wedge, E^\wedge)$ to represent a DE-HSM model that we find, i.e., a model in which:

- i. For each element $S^\wedge_i \in S^\wedge = \{S^\wedge_i, i = 1, 2, \dots, n^\wedge\}$, $S^\wedge_i$ is a subset of vertices in S , $S^\wedge_i \subseteq S$.
- ii. For each element $e^\wedge_k \in E^\wedge = \{e^\wedge_k, k = 1, 2, \dots, m^\wedge\}$, $e^\wedge_k$ is an edge in E , such that each edge $e^\wedge_k = (s_x, s_y)$ corresponds to an edge $(s_x, s_y) \in E$, wherein $s_x \in S^\wedge_i$, $s_y \in S^\wedge_k$, $i \neq k$, $i = 1, 2, \dots, n^\wedge$ and $k = 1, 2, \dots, m^\wedge$.
- iii. $S^\wedge_i \cap S^\wedge_k = \emptyset$, i.e., $S^\wedge_i$ and $S^\wedge_k$ are mutually exclusive
- iv. For each $S^\wedge_i$, $i = 1, 2, \dots, n^\wedge$, vertices in $S^\wedge_i$ are vertices of a subgraph $G'_i = \{S'_i, E'_i\}$, which is a subgraph of G , $G'_i \subseteq G$, such that $S'_i = S^\wedge_i$, i.e., vertices of G'_i and G'_i are the same, and the set of edges E'_i includes all those edges $e = (s_x, s_y) \in E$, where $s_x, s_y \in S^\wedge_i$ and $S^\wedge_i = S'_i$; i.e., $s_x, s_y \in S^\wedge_i$ and the edges are internal to G'_i .
- v. $(\cup S^\wedge_i, i = 1, 2, \dots, n^\wedge) = S$: Union of all $S^\wedge_i$ is S , i.e., each vertex in S appears in exactly one of the $S^\wedge_i$.
- vi. The following properties apply to the sets of edges in E :
 - o $E^\wedge \cap (\cup E^\wedge_i, i = 1, 2, \dots, m^\wedge) = \emptyset$; i.e., $E^\wedge$ intersect (union of all edges E'_i) is empty. Edges in $E^\wedge$ form interconnections amongst the subgraphs G'_i , where each subgraph G'_i represents a DE-FSM sub-model.
 - o Each edge $e \in E$ is either in $E^\wedge$ or in the union of all edges in E'_k , $k = 1, 2, \dots, m^\wedge$.

In other words, each $S^\wedge_i$ is a set of vertices of a subgraph $G'_i = (S'_i, E'_i)$, where $S'_i = S^\wedge_i$ forms a set of vertices of a sub-model of the DE-HSM model, and edges in E'_i are all those edges $(s_x, s_y) \in E$ that are internal edges of the subgraph G'_i . Thus, each $S^\wedge_i \in S^\wedge$ identifies a DE-HSM sub-model and edges in $E^\wedge$ represent the interconnection of the DE-HSM sub-models. Furthermore:

- o G'_i must be a connected graph.

- As a sub-model must have only one entry and one exit node to be represented by an HSM or an FSM, its subgraph can only have one entry and one exit vertex.

2) Finding a DE-HSM multi-modal model

Given a BPMN diagram, we represent it by a DAG $G = (S, E)$ for which we wish to find its equivalent DAG $G^\wedge = (S^\wedge, E^\wedge)$ in which each node $S^\wedge_i, S^\wedge_i \in S^\wedge$, represents a subset of nodes of S that are members of a subgraph $G^\wedge_i = (S^\wedge_i, E^\wedge_i)$, such that:

- Each $G^\wedge_i$ represents a sub-model $G^\wedge_i = (S^\wedge_i, E^\wedge_i)$ of $G^\wedge$ and hence $G^\wedge_1, G^\wedge_2, \dots, G^\wedge_n$ are such that
 - $G^\wedge_i \subset G, S^\wedge_i \subset S$
 - $S^\wedge_i \cap S^\wedge_k, i \neq k$, i.e., the sets $S^\wedge_i, i = 1, 2, \dots, n^\wedge$, are mutually exclusive
 - $S = \bigcup S^\wedge_i, i = 1, 2, \dots, n^\wedge$; vertices in subgraphs cover the whole graph S
 - For each edge $e = (s_x, s_y) \in E$:
 - $e \in E^\wedge$, i.e., it is one of the interconnections of the sub-models, or
 - $e \in E^\wedge_i$, for some i and where $E^\wedge_i \cap E^\wedge = \emptyset, i = 1, 2, \dots, n^\wedge$, i.e., intersection of the set of edges that interconnects the sub-models with any of edges of the subgraphs is empty
- $G^\wedge_1, G^\wedge_2, \dots, G^\wedge_n$ are such that for $G^\wedge_i$ and $G^\wedge_j$, where $G^\wedge_i$ and $G^\wedge_j$ are both subgraphs $G^\wedge$, are either
 - disconnected, i.e., not connected to each other directly (but connected indirectly via transitivity through some other $G^\wedge_k \in G^\wedge$), or are
 - connected by one edge $e = (s_x, s_y)$, where $s_x \in S^\wedge_i$ and $s_y \in S^\wedge_k, i \neq k$
- Each sub-model has exactly one entry point and exit point with the following two exceptions:
 - The subgraph that contains the start event node of the original BPMN graph does not have any entry node as there is no incoming edge from the subgraph's external nodes to any of the subgraph's internal node.
 - The subgraph that contains the end event node of the original BPMN graph does not have any exit node as there is no outgoing edge from any of the subgraph's nodes to any of the subgraph's external nodes.

The conditions in (i) ensure that the subgraphs are mutually exclusive and that they cover the whole BPMN graph. The condition (ii) states that each subgraph is connected to some other subgraph by exactly one edge. The last condition, condition (iii), is due to the fact that each DE-HSM sub-model accepts input fed to its FSM that produces output upon firing. Or rephrasing it, a sub-model has an FSM and the sub-model's input forms the input to its FSM. After FSM fires, its output forms the sub-model's output. As a consequence, each subgraph $G^\wedge_i$, which represents a sub-model in $G^\wedge$, must be such that it has one vertex, subgraph's entry node/vertex, that receives the FSM's input and one node/vertex, subgraph's exit point, that forwards the FSM's output for further processing after exiting out of the subgraph.

3) Independent subgraphs

We now present the concept of *independent* subgraphs.

Definition: Let G be a graph, $G = (S, E)$. A subgraph $G' = (S', E')$, $G' \subset G$, is called an *independent subgraph* if it satisfies the following properties:

- All nodes of the subgraph G' , except for its entry and exit nodes, have only internal edges (x, y) , $x, y \in S'$.
- There exists exactly one node/vertex, s_{en} , in the subgraph such that, in addition to its internal edges, it has only one vertex with incoming edges from external nodes – such a node is referred to as an *entry* node. (An *entry* node does not have any outgoing edges to external nodes.) The exception is that node of G' that represents the *Start* event BPMN element does not have any incoming edges from external nodes.
- There exists exactly one node/vertex, s_{ex} , in the subgraph that, in addition to its internal edges, has any outgoing edges to any external nodes – such a node is referred to as an *exit* node. (An *exit* node does not have any incoming edges from external nodes.) The exception is that node of G' that represents the *END* event BPMN element does not have any outgoing edges to external nodes.

In the above definition, except for the entry and exit nodes, all internal nodes of an independent subgraph have edges connected only to other internal nodes. As the graph S represents the control flow, and the subgraph S' has only one entry and one exit node, that means that once the initial state of the subgraph is reached during execution, the execution will proceed locally within the subgraph S' until execution reaches the subgraph's exit vertex, s_{ex} . And, that is exactly the property that is required for a DE-FSM sub-model. We now use Theorem 1, presented in [39, 40], to show the properties of independent subgraphs.

Theorem 1: Consider a BPMN model, which is represented as a DAG $G = (S, E)$, and that has two *independent subgraphs* $G_1 = (S_1, E_1)$ and $G_2 = (S_2, E_2)$, where $S_1 \subset S$ and $S_2 \subset S$ and where $|S_1| > 1$ and $|S_2| > 1$. Then one of the following holds:

- $S_1 \subset S_2$ or $S_2 \subset S_1$

- b) S_1 and S_2 are such that they either
- (i) share nodes that form an independent subgraph or
 - (ii) they share at most one edge and, furthermore, that edge is such that it is an outgoing edge from an exit node of one of the subgraphs and, furthermore, that edge is also an incoming edge to the entry node of the other subgraph.

Proof: The proof can be found in pages 39-41 of [41].

We will be using the concept of an independent subgraph for determining the DE-HSM model. However, a DE-HSM model may be such that it contains a number of other DE-HSM models that are also independent subgraphs. We are searching for independent subgraphs such that each one is represented by DE-FSM sub-model, that is a subgraph that does not have any further internal sub-models. However, we do not wish to decompose unnecessarily.

Definition: Let $G = (S, E)$ be a DAG. A subgraph G' , $G' \subset G$, is called a *Smallest Independent (SI) subgraph* if it has more than one vertex and it *does not have any proper subset that is also an independent subgraph*.

We can use SI subgraphs to form the sub-models, however, such a decomposition breaks up sub-models when there is no need for further decompositions. Consider Fig. 5, which shows an independent subgraph G' , of a part of a BPMN graph of $G = (S, E)$. $G' = (S', E')$, where $S' = \{s_{en}, v_{11}, v_{12}, v_{21}, v_{31}, v_{32}, v_{33}, s_{ex}\}$, $E' = \{(s_{en}, v_{11}), (s_{en}, v_{21}), (v_{11}, v_{12}), (v_{12}, s_{ex}), (s_{en}, v_{21}), (v_{21}, s_{ex}), (s_{en}, v_{31}), (v_{31}, v_{32}), (v_{32}, v_{33}), (v_{33}, s_{ex})\}$. However, G' has two independent subgraphs $G_1 = (S_1, E_1)$ and $G_2 = (S_2, E_2)$, where:

- i. $G_1 = \{v_{11}, v_{12}, v_{21}, s_{ex}\}$ and $E_1 = \{(s_{en}, v_{11}), (v_{11}, v_{12}), (v_{12}, s_{ex}), (s_{en}, v_{21}), (v_{21}, s_{ex})\}$
- ii. $G_2 = \{v_{21}, v_{31}, v_{32}, v_{33}, s_{ex}\}$ and $E_2 = \{(s_{en}, v_{21}), (v_{21}, s_{ex}), (s_{en}, v_{31}), (v_{31}, v_{32}), (v_{32}, v_{33}), (v_{33}, s_{ex})\}$

Clearly, it is desirable to have only one DE-HSM model representing the subgraph G' as opposed to the two sub-models, one for G_1 and one for G_2 . Further decomposition of G' into G_1 and G_2 is not only unnecessary, but it would also result in more complex interconnection of the sub-models then is necessary. Another example is an independent subgraph $G'' = (\{v_{31}, v_{32}\}, \{(v_{31}, v_{32})\})$ or an independent subgraph $G''' = (\{v_{31}, v_{32}, v_{33}\}, \{(v_{31}, v_{32}), (v_{32}, v_{33})\})$. They are both independent subgraphs, but clearly, there is no need to have separate sub-model to represent them.

We note that for any smallest independent subgraph $G' = (S', E')$, any outgoing edge from the entry vertex, s_{en} , identifies a path of connected nodes, such that the path terminates at the exit node, s_{ex} . Furthermore, any node on that path has only two edges that are on the path from s_{en} to s_{ex} and no other edges. If a node, say a node $s_1 \in S'$, which is on a path from s_{en} to s_{ex} , did have another edge, say to a node $s_2 \in S'$, that is not on the same path from s_{en} to s_{ex} , it would violate the properties of the smallest independent subgraphs as either: (i) If $s_2 \notin G'$, then an edge from the node s_1 to s_2 within the subgraph G' leads to the node s_2 that is outside the subgraph G' , which means that the subgraph is not an independent subgraph as it has two nodes with edges outgoing to nodes that are external to the subgraph. Or (ii) $s_2 \in G'$, but then the edge (s_1, s_2) would create an independent subgraph within the smallest independent subgraph G' , which is a contradiction.

4) Largest Smallest Independent (LSI) subgraph

Consequently, we wish sub-models to represent independent subgraphs such that each subgraph does not have any independent subgraph itself unless that independent subgraph (i) has the same entry and exit nodes as its parent graph or (ii) that independent subgraph is fully on a path from the subgraph's entry node to its exit node. We introduce the concept of the *Largest Smallest Independent (LSI) subgraph*.

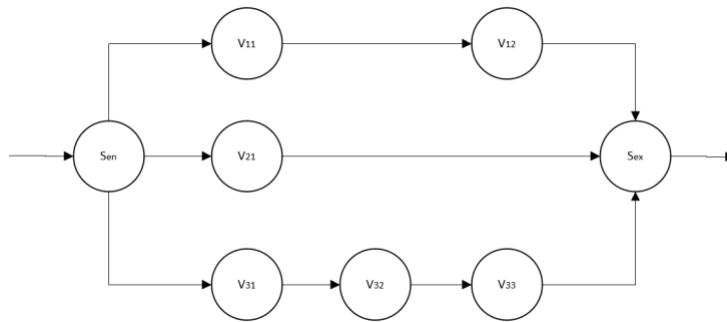


Fig. 5. Independent subgraph with proper subsets that are also independent subgraphs

Definition: Let G be a graph, $G = (S, E)$. A subgraph $G' = (S', E')$, $G' \subset G$, is called a *Largest Smallest Independent (LSI) subgraph* of G , if it is an *independent subgraph* and, furthermore, any vertex $s \in S'$, which is neither an entry nor an exit node of an independent subgraph, has exactly one incoming and one outgoing edge.

We will use the properties of the LSI subgraphs to construct, from the given graph G , an equivalent representation $G^\wedge = (S^\wedge, E^\wedge)$, where each $S^\wedge_i \in S^\wedge$ is a set of vertices of a subgraph G'_i , $i = 1, 2, \dots, n^\wedge$. Each subgraph G'_i is an LSI subgraph that represents a DE-FSM sub-model, and $E^\wedge$ is a set of edges in E that represents interconnections of the sub-models represented by the subgraphs.

5) Algorithm to find LSI subgraphs

The previous sections dealt with formalities and abstractions suitable for presentation of the concepts of independent and LSI subgraphs. We now provide a pseudocode of an algorithm to find LSI subgraphs for a given DAG $G = (S, E)$, which represents a BPMN diagram. The algorithm, called *findLSISubgraphs*, has as its input a global DAG $G = (S, E)$, which represents a BPMN diagram. The method results in a DAG $G^\wedge = (S^\wedge, E^\wedge)$, such that $E^\wedge$ represents edges and each $S^\wedge_i \in S^\wedge$ represents a subset of nodes S'_i of the graph G'_i , where the nodes of S'_i , $i = 1, 2, \dots, n^\wedge$, form a subgraph $G'_i = (S'_i, E'_i)$, where $S'_i = S^\wedge_i$ is a set of nodes representing an LSI subgraph that is an FSM sub-model. Furthermore, E'_i represents interconnections of the nodes in S'_i , wherein each edge $e = (s_x, s_y)$ is an internal edge to S'_i , i.e., where $s_x \in S'_i$ and $s_y \in S'_i$. Thus, each $S^\wedge_i$ defines an LSI subgraph $G'_i = (S'_i, E'_i)$, while the set of edges $E^\wedge$ of $G^\wedge$ form interconnections amongst the LSI subgraphs.

The method uses a global variable $G = (S, E)$ to mark the nodes with information on vertices incoming and outgoing degrees, where outgoing degree of a vertex s is the number of directed edges of the form (s, s_x) , for some vertex $s_x \in G$, and an incoming degree is the number of edges of the form (s_x, s) , where $s_x \in G$.

Recall from the section III-B, which dealt with preprocessing of the BPMN graph, that preprocessing replaces any gate that is both a join gate and a fork gate with an equivalent representation by one merge gate and one fork gate, wherein the merge gate has one or more incoming edges but only one outgoing edge that is also the only incoming edge to a fork gate that has only one has more than one outgoing edge.

Before we proceed with the presentation of the algorithm to find LSI subgraphs, we make a few observations about LSI subgraphs:

- With the exception of the entry of exit nodes, there is no vertex that has both incoming and outgoing degrees being greater than one.
- If a vertex has an incoming degree of more than one, then it is a merge/join gate with one outgoing edge. In such a case, the vertex may only be an exit node from an LSI subgraph.
- If a vertex has an outgoing degree of more than one, then it is fork/split gate, with an incoming degree of one. In such a case, the vertex may only be an entry node to an LSI subgraph.
- An internal node of an LSI subgraph has an incoming degree and outgoing degree being exactly one.

The algorithm/method *findLSISubgraphs*, shown in Fig. 6, has a DAG $G = (S, E)$ as its input. Recall that the *start event* has only one edge, which is an outgoing edge. Each node $s \in S$ is assumed to have the following attributes:

- s.inDegree*** ... the number of edges that are incoming into the vertex s
- s.outDegree*** ... the number of edges outgoing out of the vertex s
- s.root*** ... reference to a node that may be an entry of an LSI subgraph to which the node s may belong

The method *findLSISubgraphs ()* uses the following auxiliary methods:

- vertex x = nextInSearchOrder (G)*** ... the method facilitates a breadth-first search traversal of the connected DAG G , in which that vertex $s \in S$ that represents the BPMN *START event* is the start of the traversal. Each of the other nodes has at least one incoming edge and one or more outgoing edges, with the exception of the vertex that represents the BPMN *End event*, which does not have any outgoing edge. Calling the method *nextInSearchOrder(G)* repeatedly results in the method returning a reference to the next node in the breadth-first search traversal of G and returning null once all vertices have been traversed.
- LSISubgraph G'_i = isLSISubgraph (s, d)*** ... given vertices s and d , the method determines whether the vertices s and d identify an LSI subgraph by checking the cases that are discussed below. If it does discover an LSI subgraph, it is returned by the method; otherwise, the method returns null. We first discuss the special cases that exist for the vertices that represent the *START* and *END* BPMN elements.
 - If the in-degree of the vertex, say s , is zero, then the vertex represents a *START* BPMN element and hence it cannot be an exit node. The *s.root* attribute is set to s as it will be the entry node of an LSI subgraph, which is also the root node of the subgraph.

- If the out-degree of vertex is zero, then the vertex represents the *END* BPMN element. It is treated as an exit vertex of an LSI subgraph.
- $vertex\ s_x = pointedToBy(s) \dots$ The method returns null if the vertex s does not have any incoming edges or if it has more than one incoming edge. Otherwise, it returns that vertex s_x that is the source for the only edge leading to s , i.e., it returns that vertex s_x or for which there exists an edge $e = (s_x, s)$.

The algorithm uses a breadth-first search traversal of the DAG G , starting with that graph node s that represents the BPMN element that is the *Start* event, while its output has been described already. We now discuss the details of the `findLSISubgraphs()` algorithm that is shown in Fig. 6.

The first part deals with initialization that sets the attributes of each node of the DAG G , namely attributes $s.inDegree$, $s.outDegree$, and $s.root$. Following that there is a while loop that is used to determine whether the node s , returned by the `nextInSearchOrder( $G$ )`, is an exit node of an LSI subgraph and whether that node identifies an LSI subgraph, which has been found and should be included in the set of LSI subgraphs output by the method, or whether it is some other case and how it should be handled. These cases are now discussed by making specific references to the lines of the algorithm presented in Fig. 6.

The line 18 within the while loop checks the indegree of the vertex s . If the indegree of the vertex s is zero, then the vertex s represents a *START* BPMN element and hence it is an entry node of an LSI subgraph. If it is, then the $s.root$ attribute is set to s as it will become a root node of an LSI subgraph. The while iteration is exit and another iteration starts (lines 24-26). Note that for the vertex representing the *START* BPMN element, its outdegree cannot be greater than one as only a fork gate can have an out-degree greater than one.

- If the $s.inDegree > 1$ (line 23), then the vertex s is an exit node and an LSI subgraph has been discovered and added to the set of LSI subgraphs $G^\wedge$, and the loop is exited (lines 24-26)
- If the $s.inDegree$ is equal to 1 (line 27), then the node's outdegree is checked.
 - If the outdegree of the vertex s is zero (line 28), then the vertex s represents the *END* BPMN node and, consequently, the LSI subgraph identified by the entry and exit nodes $s.root$ and s , respectively, is returned as an LSI subgraph G' that will be added to the set of LSI subgraphs $G^\wedge$ (lines 29-31).
 - If the outdegree of the vertex s is equal to one (line 32), then it cannot be determined as yet whether the node s is or is not an exit node of an LSI subgraph. The vertex s could be an exit node such that there is only a single path to the node s , while the out-going edge from s , edge (s, s_y) , leads to a node that has an out-degree of greater than one and hence is the entry node of another LSI subgraph, in which case the node s is an exit node from an LSI subgraph. If the outdegree of the node s_y is one, then the node s is not an exit node from an LSI subgraph as the node s_y could be safely added to the LSI subgraph identified by the vertices $s.root$ and s without violating the LSI subgraph properties. Thus, to determine whether or not s is an exit node, we must examine the out-degree of the node s_y , where s_y is the destination node of the outgoing edge from s (line 33):
 - If the outdegree of the vertex s_y is greater than zero (line 34), then the vertex s_y , where s_y is the destination vertex of the out-going edge from s , i.e., the edge (s, s_y) , is the entry node of the next LSI subgraph and hence the node s is the exit node of the LSI subgraph. The found LSI subgraph is added to the set of LSI subgraphs that is output (lines 35-37).
 - If the outdegree of the vertex s_y is equal to 1 (line 39), then the vertex s is neither exit nor entry node and the $s.root$ attribute is set to refer to the $s.root$ attribute of the vertex that has an outgoing edge to points to s .
 - As a note, if the outdegree of the vertex s_y is equal to 0, then s_y represents the *END* BPMN element, in which case nothing needs to be done as the node s_y will be processed on a subsequent iteration of the while loop.

Once all nodes are processed, the algorithm performs a post-processing step that ensures that any LSI subgraphs that were discovered and added to the set $G^\wedge$ and that have the same entry and exit nodes are combined into one LSI subgraph. For instance, if the algorithm outputs two separate independent subgraphs G_1 and G_2 , where G_1 and G_2 are:

- $G_1 = \{v_{11}, v_{12}, v_{21}, s_{ex}\}$ and $E_1 = \{(s_{en}, v_{11}), (v_{11}, v_{12}), (v_{12}, s_{ex}), (s_{en}, v_{21}), (v_{21}, s_{ex})\}$
- $G_2 = \{v_{21}, v_{31}, v_{32}, v_{33}, s_{ex}\}$ and $E_2 = \{(s_{en}, v_{21}), (v_{21}, s_{ex}), (s_{en}, v_{31}), (v_{31}, v_{32}), (v_{32}, v_{33}), (v_{33}, s_{ex})\}$

Then the post-processing step combines all independent subgraphs that have the same entry and exit nodes into one subgraph. For the example of G_1 and G_2 , the post-processing step combines the two sub-graphs into one as is shown in Fig. 5.

Algorithm 1: findLSIsubgraphs

```

(1) Input: DAG  $G = (S, E)$ ;
(2) Output: setOfLSI subgraphs  $G^* = \{ G'_i = (S'_i, E'_i), i = 1, 2, \dots \}$ 
(3)           where each  $S'_i$  corresponds to the  $S_i^{\wedge}$  an element of  $S^{\wedge}$ , where  $G^{\wedge} = (S^{\wedge}, E^{\wedge})$ ,
(4)           and  $E^{\wedge}$  is the interconnection of the LSI subgraphs  $G'_i, i = 1, 2, \dots$ 
(5) BEGIN
(6) Initialization;
(7) for each node  $s$  in  $S$  do
(8)   Let  $s.inDegree$  = the number of incoming edges to  $s$  ;
(9)   Let  $s.outDegree$  = the number of outgoing edges from  $s$  ;
(10)  Let  $s.root = s$  ;
(11) end
(12) Let  $G^*$  be empty;
(13) var vertex  $s, s_x, s_y$ ; var edge  $e = \{s_x, s_y\}$ ; var LSIsubgraph  $G' = (S', E')$  ;
(14)
(15) // invocation of the breadth-first search traversal of the DAG  $G$  ;
(16) while (  $s = nextInLevelOrder(G) \neq nil$  ) do
(17)   // Check if vertex  $s$  represents the START BPMN element
(18)   if ( $s.inDegree = 0$ ) then
(19)     let  $s.root = s$  ; //Assumption: BPMN diagram has more than the START and END element
(20)     exit while loop;
(21)   end
(22)   //  $s$  may be an internal LSI subgraph node or an exit node or may represent END element;
(23)   if ( $s.inDegree > 1$ ) then //  $s$  node is an exit node from an LSI subgraph
(24)      $G'_i = isLSIsubgraph(s.root, s)$ ; // Should return an LSI subgraph
(25)     add  $G'_i$  to  $G^*$ ; //  $G'_i$  should not be null
(26)     exit while loop; // exit the current iteration of the while loop and check for the next iteration
(27)   if ( $s.inDegree = 1$ ) then
(28)     if ( $s.outDegree = 0$ ) then //  $s$  represents the END BPMN element => it is exit node
(29)        $G'_i = isLSIsubgraph(s.root, s)$ ; // Should return an LSI subgraph
(30)       add  $G'_i$  to  $G^*$ ; //  $G'_i$  should not be null
(31)       exit while loop; // exit the current iteration of the while loop and check for the next iteration
(32)     elseif ( $s.outDegree = 1$ ) then // there is only one edge from  $s$  and  $s$  may or may not be an exit node
(33)       Let  $e = (s, s_y)$ ; // Find the destination vertex  $s_y$  of the only outgoing edge from  $s$ 
(34)       if ( $s_y.inDegree > 1$ ) then //  $s$  is an exit node from an LSI subgraph as  $s_y$  is an entry node
(35)          $G'_i = isLSIsubgraph(s.root, s)$ ; // Should return an LSI subgraph
(36)         add  $G'_i$  to  $G^*$ ; //  $G'_i$  should not be null
(37)         exit while loop; // exit the current iteration of the while loop and check for the next iteration
(38)       else
(39)         if ( $s_y.inDegree = 1$ ) then //  $s$  is neither exit nor entry node
(40)            $s.root = (pointedToBy(s)).root = s$  ;
(41)         endif // if ( $s_y.inDegree = 0$ ) then * The node  $s_y$  represents the END BPMN that will be
(42)           recognized as such and processed on the next iteration */
(43)       endif
(44)     endif
(45)   endif
(46) endwhile;
(47) // Post processing:
(48) Combine any subgraphs  $G'$  in  $G^*$  that have the same entry and exit nodes into one subgraph;
(49) END;

```

Fig. 6. Algorithm *findLSIsubgraphs()*

The significance of decomposing the BPMN model into DE-HSM sub-models that satisfy the property of LSI subgraphs is that the LSI subgraphs lead to sub-models that can be modeled directly by a DE-FSM model, i.e., they do not need any further decomposition into sub-models.

6) Example

Consider the sample supply-chain BPMN model shown in Fig. 3 that was also used in [16]-[18]. Applying the algorithm *findLSISubgraphs()* results in the LSI subgraphs, listed by the method *foundSubgraph*, that are shown in Fig. 7 and are shown below.

- S_1 ... This subgraph contains INIT, Buyer sends offer, Manufacturer receives order, calculate demand, Manufacturer places order and Middleman receives order.
- S_2 ... This subgraph contains *forward order*, *order transport*, *supplier receives order*, *produce*, *order transport*, *prepare transport*, *carrier receive order*, *supplier receive request and request details*.
- S_3 ... This subgraph contains *provide details*, *provide waybill*, *receive details*, and *receive waybill*.
- S_4 ... This subgraph contains *carrier deliver order*, *manufacture receives order*, *report start of production*, *manufacturer deliver product*, *buyer receives product*, and *SUCCESS*.

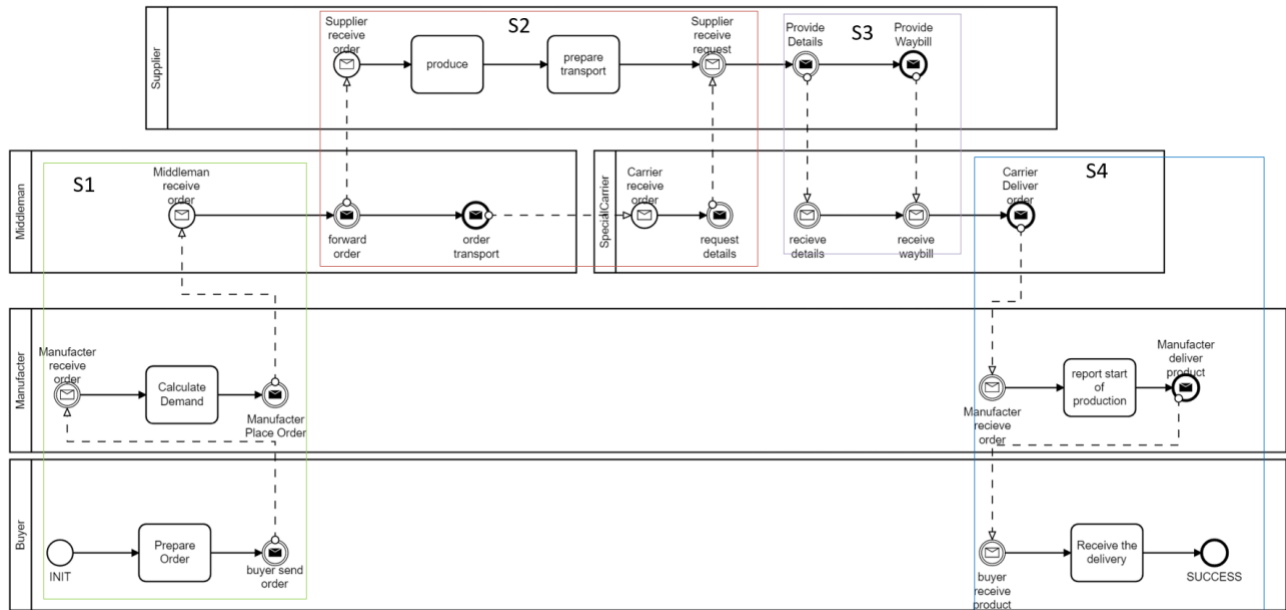


Fig. 7. BPMN model and found subgraphs

To explore how the algorithm works, consider its execution after its initialization when the while loop is executed. The breadth-first search traversal returns the *INIT* node as the first node. Thus the node s in the while loop refers to the *INIT* BPMN element on the first iteration of the loop, at which point $s.root = s$, which represents the fact that the s node will be the entry node of the LSI subgraph that will be discovered. Following the first loop, subsequent loops retrieve nodes: *Prepare Order*, *buyer send order*, *manufacturer receive order*, and *Calculate Demand*. All the listed vertices have exactly one incoming and one outgoing edge and thus cannot be exit nodes of an LSI subgraph. As a result, their processing by the while statement will set the attribute $s.root$ to refer to the vertex representing the *INIT* node, as set by the line (39) – that vertex will become a root of an LSI subgraph.

Assume that all nodes from the node *INIT* to *Calculate Demand* have been processed by the while loop and now the reference to the node *Manufacturer Place Order* has been returned by the method in the while statement *while s = (nextInSearchOrder(G)) ≠ nil do*, and that the reference to that vertex has been stored in the variable s . Since both the indegree and outdegree are one for the node s , the lines (33) to (40) of the method in Fig. 6 are executed next. As the outdegree of the node s_y , where the vertex s_y is the destination vertex of the edge (s, s_y) , is also 1, the vertex s is neither an entry node nor an exit node and hence the attribute $s.root$ is set to $s.root = (pointedToBy(s)).root = s$ – as was done by while loop processing of the previous vertices.

On the next iteration of the while loop, the node returned by the breadth-first search method is the node s that represents the *Middleman receive order* BPMN element. As both the in and out degrees are equal to 1, the lines (33) to (40) are executed. As the outdegree of the node s_y , where the vertex s_y (represents the *forward order* BPMN element) is the destination vertex of the

edge (s, s_y) , is greater than 1, then the vertex s_y is the entry node of the next LSI subgraph, and hence the node s is the exit node of the subgraph. Consequently, the statements in lines (35) to (37) are executed and an LSI subgraph, with the entry node being the vertex representing the *INIT* BPMN element and the exit node being the *Middleman receive order* BPMN element.

E. Building DE-FSM Sub-models and Their Interconnection

Thus far, we described how to determine a DE-HSM model, that is, how to find a DE-HSM model for which each of its sub-models does not contain any further sub-models. In this section we describe how to build each of the DE-FSM sub-models and interconnect them.

1) Building a DE-FSM submodel

The previous section described how to find a DE-HSM model for a given graph G that represents the BPMN model. The DE-HSM model is represented by a graph $G^\wedge = (S^\wedge, E^\wedge)$, such that each $S^\wedge_i \subset S^\wedge$ represents a subset of nodes of the graph G , where the nodes of $S^\wedge_i$, $i = 1, 2, \dots, n^\wedge$, are also the nodes of a subgraph $G^\wedge_i = (S^\wedge_i, E^\wedge_i)$ of G , where $S^\wedge_i = S^\wedge_i$ is a set of nodes representing an LSI subgraph that is an FSM sub-model.

Given an LSI subgraph $G' = (S', E')$ that represents a DE-FSM sub-model, we build its model using a breadth-first search traversal of its graph $G' = (S', E')$, starting at its entry node s_{en} . Recall that each sub-model represents a DE-FSM model that uses one DE queue, which is shared by all sub-models, and an FSM that represents the functionality. Also recall from the previous section that described how to find the DE-HSM model, that a fork gate denotes an entry node of an LSI subgraph, while a merge gate is an exit node/vertex from an LSI subgraph. However, an LSI subgraph is such that each of its vertices that is neither the entry nor the exit vertex of the subgraph has exactly one incoming and one outgoing edge. Thus, with the exception of its entry or exit nodes, there are no other vertices that represent a gate. As a consequence, the translator creates the sub-model in two phases.

The first phase corresponds to the flow control entering the sub-model during the execution phase, that is when the flow control reaches the subgraph's entry node. Depending on the type of the entry node, further execution may proceed as one processing stream for an exclusive fork gate, or more than one processing streams in case of an inclusive fork gate. Note that an exclusive or inclusive fork gate may have a guard along each outgoing edge, wherein a guard along an outgoing edge represents the constraint that the FSM's input must satisfy for the execution to proceed along that edge. Of course, for an exclusive gate, guard constraints must be such that only one evaluates to true at any one time. Thus, an inclusive fork gate also covers a parallel fork gate when all guards are set to true.

If the entry node of the LSI subgraph represents an exclusive fork gate BPMN element, then in the first phase, the root of the FSM is created. For an exclusive fork gate, in addition to the root of the FSM, additional states are appended for a binary search that will result in identifying a path that represents the control flow along each of the paths from the entry node to the exit node. For instance, if there are four outgoing edges from an exclusive fork gate with the guards being A, B, C, and D, then Fig. 8 shows the vertices added to the FSM in order to identify the path for each of the outgoing edges to the exit node. In the second phase, on each iteration of the search, a new state is added on a path for which the input satisfies the guards. In short, there is one FMS for an entry node that represents an exclusive fork gate, such that the FSM's root is followed by transitions that result in n paths, one path for each of the outgoing streams from the gate. In the subsequent phase, state transitions are built along the outgoing paths, each one representing functionality for one of the guards on the exclusive fork gate.

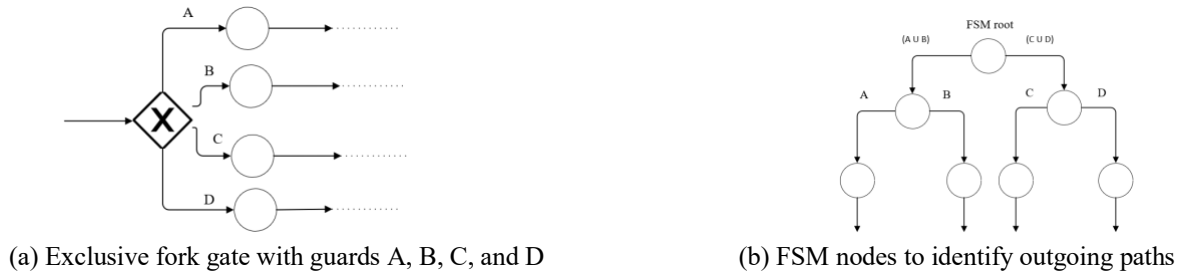


Fig. 8. Entry node that represents an exclusive fork gate and FSM transitions as paths of FSM

If the entry node represents an inclusive fork gate BPMN element, then the translator creates $n+1$ FSMs if there are n outgoing streams out of the inclusive fork gate. In addition, one FSM, referred to as a *control FSM*, is created to determine at the execution time, to which of the outgoing streams from the fork gate does the input to the FSM belong when an FSM fires, i.e., when it receives input to which it needs to react. The controlling FSM is used in the execution phase to determine to which of the FSMs should the input be directed depending on the evaluation of the guards – the identification of the FSMs for which guards evaluate to true is the output of the controlling FSM's firing. In addition, when the exit from the firings of the controlling

FSM is reached, the flow of control is directed towards each of the concurrent FSMs for which the guard evaluated to true. This is achieved the monitor component at run-time by creating a DE for each of the guards that evaluates to true. For each one, the monitor inserts a DE event that causes it to fire and hence causes execution of the firing of the corresponding concurrent FSM when the event reaches the head of the queue. Fig. 9 shows the inclusive fork gate and the resulting control FSM and the concurrent FSM.

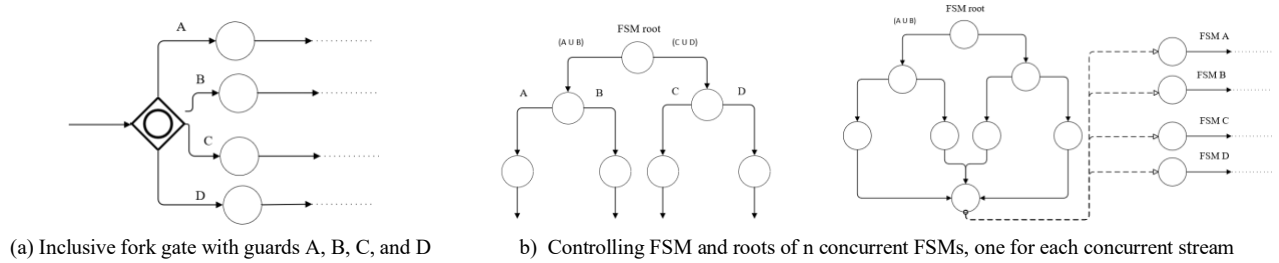


Fig. 9. Entry node that represents an inclusive fork gate and control FSM and one FSM for each of the concurrent process streams

Once the FSMs are prepared, a breadth-first search of the nodes of the subgraphs is used to build the transitions states of the concurrent FSMs. For an exclusive fork gate, all transitions are in the one FSM along the paths from the root to the exit node, wherein the initial vertices are created by the translator in the initial phase to guide the input to the appropriate path using the guard conditions. For an inclusive fork gate, for each node of the breadth-first search, the translator first determines to which of the FSMs the node belongs and then the translator augments the corresponding FSM with the state transition for the node. Both phases are now described.

Initial phase – Entering a sub-model

The translator examines the BPMN element that the entry vertex of the sub-model represents, and the transformation into a sub-model depends on whether the BPMN element is not a gate, is an exclusive fork gate, or is an inclusive fork gate:

- The entry node is not a gate. The processing within the sub-model is represented by a single stream with the functionality represented by an FSM. The translator builds the root of the FSM, with the rest of the FSM states created by examining each the nodes of the subgraph as described below in the main phase that builds the FSM transitions.
- The entry node is an exclusive fork gate. The processing within the sub-model is represented by a single stream with the functionality represented by one FSM. The translator builds the root of the FSM, and then it inserts states that check the input and perform binary search using guards of the exclusive gateway to determine on which path, from the entry node to the root, is a new transition inserted (as shown in Fig. 8).
- The entry node is an inclusive fork gateway. The processing within the sub-model is represented by multiple concurrent streams by creating one *concurrent FSM* for each of the outgoing concurrent stream outgoing from the entry node that represent the inclusive fork gate BPMN element. In addition, the translator the *control FSM* that is used to direct input coming into the sub-model into an appropriate FSM for an outgoing flow-control stream determined by examining the input to the FSM using constraints represented by the guards on outgoing edges of the BPMN gateway. As in the previous case of an exclusive gateway, the translator inserts into the controlling FSM states that check the input using Boolean expressions appearing in the guards of the gate that the node represents as shown in Fig. 9.

Main phase

In the main phase, subgraph vertices are processed, vertices that have exactly on incoming and one outgoing edge. Such a vertex results in a new FSM state and transition to be added to an FSM: For an inclusive fork gateway, the new state and its transition are added to the appropriate concurrent FSM, while for the exclusive fork gateway, the new state and its transition are added to the appropriate path of the one FSM that has one path, from the subgraph's entry node to its exit node, for each of the outgoing edges out of an inclusive-fork gateway.

2) Connecting the submodels

Once the sub-models are built, the overall model is built by finding that subgraph that contains the node that represents the BPMN *START* event – call it the *start subgraph*. The overall model is now built by interconnecting the sub-models. The exit node of the *start subgraph* is connected to entry nodes of *LSI subgraphs* *i*, for which there is an outgoing edge from the exit node of the *start subgraph* and to the entry node of the *LSI subgraph* *i*. For each such interconnection, the translator takes the output from the FSM of the exit node and creates a DE event that is queued into the DE queue, with the event being targeted for processing by the FSM that represents the functionality of the *LSI subgraph* *i*. This process repeats for each of the *LSI subgraphs*.

V. TRANSFORMING A DE-HSM MODEL INTO METHODS OF A SMART CONTRACT

In the following subsections, we overview the transformation of the DE-HSM model into the methods of a smart contract. We only perform an overview as our approach has been described in [32], [42]-[43] and, in some respect is similar to work of others. However, we differ in providing the developer with the functionality to deploy parts of the smart contract to be processed on a sidechain for the purposes of reducing the cost and/or to increase privacy – and those are the aspects that we highlight in more detail.

A. *Creating and Deploying Smart Contract Methods*

During the contract execution phase, user participants and their systems interact with a DApp that, in turns, makes calls to the *DApp API* that our system provides. At run-time, once a smart contract method is invoked, its input is analyzed, and it is used to create an appropriate event that is queued into the DE queue. The DE queue contains events that are continuously dequeued and processed. Processing of a DE event includes: (i) analysis of the DE event that determines which FSM is to fire and prepares input for that FSM; (ii) input is given to the FSM; (iii) FSM fires and produces output; and (iv) FSM output is transformed into a response to the smart contract method invocation. This is all performed under the control of the monitor/mediator using information prepared during the design phase.

Thus, the run-time architecture of the system is similar to other approaches in the way the system interacts with the DApp and performs mapping of the DApp interactions with API into calls to the methods of a smart contracts and how the system responds to the DApp requests. Where we depart is in how the flow control is managed.

The internal representation of the state of the system is based on the DE-HSM model that determines the flow-control between the FSM sub-models that represent the state of computation of individual DE-FSM sub-models. We faithfully model the BPMN graph using a DE-HSM model that is used to channel input into the appropriate sub-model where the input is provided to the sub-model's FSM that fires and produces output. Depending on the DE-HSM model, the output produced by the FSM may be used as input to another sub-model and the process repeats, or the output is analyzed and provided as output for the call to the smart contract method, which in turn provides its returned parameters to the DApp method that invoked the DApp API. The channeling of outputs produced by sub-models is according to the interaction between the sub-models as is defined by the DE-HSM model, while the function of each sub-model is represented by an FSM, or a group of FSMs if the sub-model has concurrent processing indicated by the inclusive fork gates, that also faithfully models a part of the BPMN graph that is a BPMN LSI subgraph.

We also depart from the work of others in that we facilitate off-chain processing on a sidechain, which is described next.

B. *Sidechain Processing for Privacy and Cost Reduction*

After the BPMN model is transformed into a DE-HSM model and individual sub-models are developed, but before the transformation into the methods of a smart contract, the developer is offered a design choice to deploy a part of the smart contract for processing on a sidechain for the purposes of privacy, reducing the processing cost, or both. To make that decision, sub-models are presented to the developer to select those that should be deployed on a sidechain. Actually, as a part of testing and before the final deployment, two versions of the smart contract are developed and deployed: In one, the sub-models selected by the developer are deployed and executed on a sidechain, while in the second version, all smart-contract methods are deployed and processed on the mainchain. Delays are measured for both versions and the results are provided to the developer so that she/he can compare the cost of the smart contract execution, on the mainchain-only versus on the mainchain with sidechain processing of sub-models selected by the developer. The developer has information on the incurred delays in both cases and, if the mainchain or sidechains use Ethereum EVM, such as Quorum does, cost estimates in ETH gas are also provided to the developer.

We first describe our system architecture for the run-time phase with sidechain processing. We then describe how we assist the developer in determining which parts of the smart construct are suitable for sidechain processing, either for privacy or for reducing cost. Examples of use of sidechains are provided in a following section that describes our *Proof of Concept (PoC)*, the TABS tool.

1) *System architecture with sidechain processing*

Fig. 10 shows the system architecture at run-time, for the mainchain and for the sidechain. Internal architecture for sidechain processing is in big part similar to that for the mainchain, which was shown in Fig. 2. Interoperability, between the mainchain and the sidechain, is achieved in part by a mainchain contract method calling a smart contract method on a sidechain. There are, however, additional issues involved, as our sidechain contract is really an integral part of the mainchain contract, but it is executed on a sidechain. When smart contract methods are moved from the mainchain to a sidechain, of course, they have to be invoked from the mainchain. When there is a call on a mainchain to a smart contract that is deployed on a sidechain, the mainchain method must re-direct the call to the sidechain. However, difficulties arise when state variables are accessed during

a method executed on a sidechain – a method deployed on a sidechain does not have access to any state variables stored on the mainchain. For that reason, they need to be fetched from the mainchain and stored on a sidechain in a cache. Consequently, on the first call to a method deployed on a sidechain, in addition to the call parameters, also included is blockchain data that will be accessed by the method. At the design time, static analysis of the sidechain code is used to determine which data is provided on the first call to a method so that it would populate the cache on a sidechain. Of course, there will still be cache misses, due to the sidechain method referencing objects dynamically.

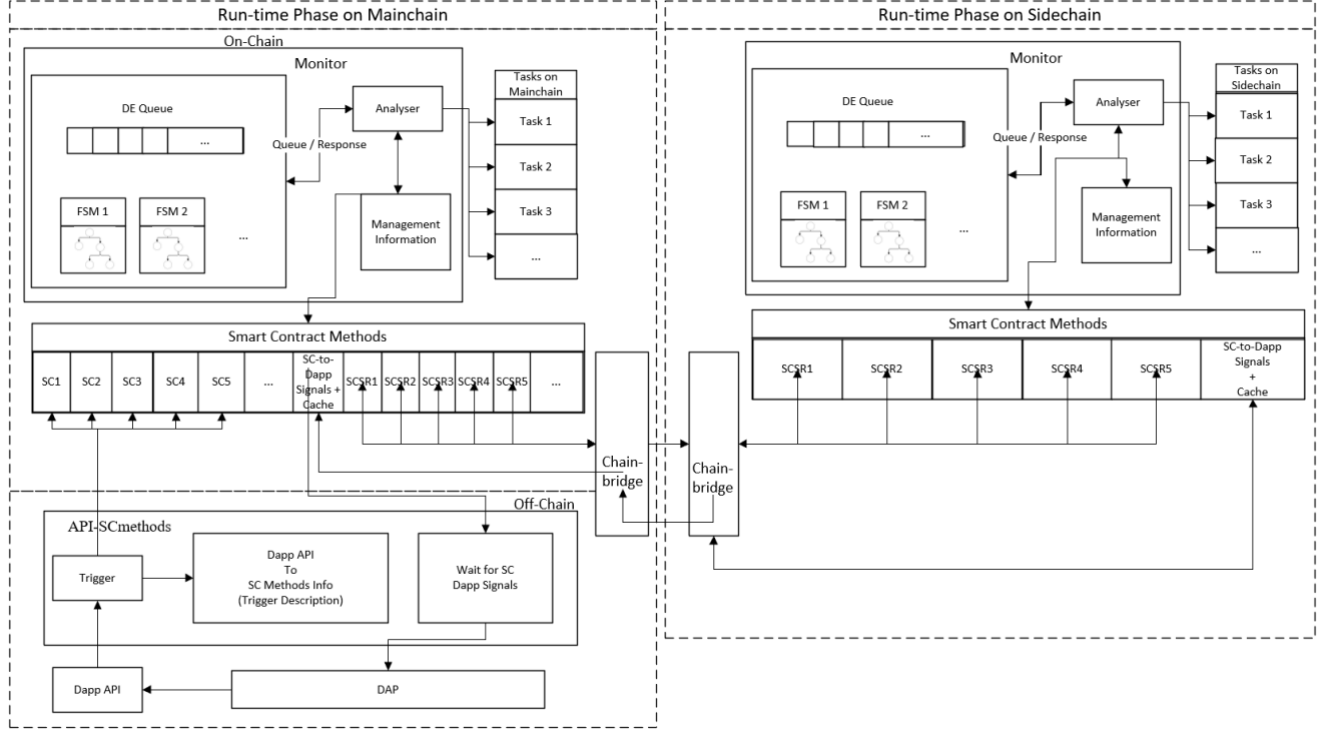


Fig. 10. System architecture with mainchain and sidechain processing

C. Sidechain Processing: What, When, How

To process a part of the smart contract off-chain, the following decisions need to be made:

- What should be processed off-chain, that is which parts, referred to as patterns, of smart contract should be processed off-chain.
- When should processing be performed off-chain, that is under which conditions should sidechain processing be used?
- How to achieve sidechain processing.

1) Identifying patterns suitable for sidechain processing

The concept of independent subgraphs, which was used to determining the DE-HSM model given a BPMN model, is useful in determining which patterns are suitable for processing off-chain. Processing patterns off-chain causes overhead, some of which is due to the sidechain not having access to the mainchain. Consequently, if a pattern interacts with the main chain methods, overhead cost of communication between the mainchain and sidechain arises. For that reason, a pattern moved to the sidechain should be independent from the other methods of the smart contract executing on the main chain. And independent LSI subgraphs have that property in that once the flow of control enters the LSI subgraph, the flow of control does not exit that subgraph until the computation in that subgraph is completed when the flow of control exits the subgraph. Thus, if the LSI-subgraph processing is deployed on a sidechain, with the exception of the main chain contract passing the application's input as parameters to the sidechain, there is no processing on the mainchain until the execution exits from the independent subgraph executed on a sidechain. As the design phase uses the concept of LSI subgraphs to find the DE-HSM model, each sub-model is represented by an LSI subgraph and hence, deploying parts of the smart contract to a sidechain is equivalent to deploying DE-HSM sub-models on the sidechain. The developer is thus provided with the choice of which DE-HSM sub-models that can be selected for processing on a sidechain.

2) *When to process on a sidechain (deciding to process a pattern off-chain)*

Sidechain processing may be cheaper and may be used to decrease the cost of processing a pattern. However, processing a pattern on a sidechain causes overhead due to interaction between the mainchain and the sidechain. Thus, from the cost perspective only, sidechain processing should be used only if its cost, including the overhead cost, is less than processing the pattern on the mainchain. We determine the cost of processing on a sidechain that also includes the overhead cost by actually doing a live test of running the DApp when (a) the smart contract is deployed the mainchain only and (b) the pattern is deployed on the sidechain and thus measuring all delays, including delays due to overhead. If processing on a mainchain or a sidechain involves EVM machine, the cost is also estimated in Eth for each of the smart contract methods and is provided to the modeler.

However, another reason to process a pattern on a sidechain may be privacy. A smart contract deployed on the mainchain is such that all participants can explore all data stored on the blockchain by the smart contract – hence there is no privacy that can be provided so that some collaboration be private, that is confidential to other actors². For instance, a subset of actors performs a local collaboration represented by a pattern and that collaboration should be private in that its details (its blockchain data) should not be visible to the other participants to the smart contract who do not participate in that collaboration. In such a case, we should process the pattern on a sidechain, which will provide privacy and the other actors will not be privy to the pattern’s processing details. Of course, all smart contract participants will be able to see any data returned by the sidechain computation that is stored on the mainchain.

3) *How to process off-chain*

Once it is determined, which pattern is to be processed off-chain, the next question is how to facilitate it. A pattern is a set of smart contract methods, such that they are invoked either due to a signal from the DApp API or because they collaborate and invoke each other, but “within” the pattern. A smart contract method executing on a sidechain is likely to need to know the state of execution and other information stored on the blockchain – however, a method on the sidechain does not have access to the mainchain and its data. Consequently, when transitioning to off-chain execution of a smart contract method, in addition to the parameters passed to the method, the system also needs to find, retrieve, and deliver to the off-chain processing any mainchain data that the methods on a sidechain read. Similarly, upon execution returning back to the mainchain, any writes to the blockchain that were performed by the off-chain execution of the pattern need to be collected and handed-off, together with transition outputs, for recording on the blockchain. Finally, upon completion of a method executed off-chain, before the blockchain is updated with the new state and data values written by the off-chain method, the results of off-chain execution must be reviewed and approved/attested by each of the participants affected by the off-chain computation. As a consequence of the above, we provide interfaces for the following three phases when methods of a pattern are deployed on a sidechain:

Pattern start: Upon the first invocation of an off-chain pattern method, our software tool prepares appropriate data structures to support on-chain / off-chain interaction. Most important is a cache for mainchain data accessed by the pattern executed off-chain. Off-chain code uses a local cache for reading and writing blockchain data – its data structure needs to be prepared. On a cache miss, data is retrieved from the main blockchain and then stored in the off-chain cache using a getter method prepared just for that purpose to retrieve the data from the mainchain. For the cache, we use IPFS as a distributed system for storing and accessing temporary files and data. When Blockchain needs to access or examine the data in the cache, TABS allocates a data resource stored on IPFS by tracking the content address of corresponding data stored on IPFS [38].

As mentioned before, a simple static analysis of the pattern methods is used to determine which state variables, that is blockchain data, should be passed to the sidechain with the first invocation of the pattern so that the cache could be populated with them. Of course, any dynamically accessed data may result in a cache miss.

Pattern middle: After the first invocation, interaction between the off-chain and on-chain communication is concerned with provision and return of appropriate parameters and data for methods executed off-chain. Also, on a cache miss to the off-chain cache, data needs to be retrieved from the main chain using a getter method in order to service the cache fault.

Pattern end: Upon completion of the last method that is executed off-chain, which occurs upon a transition from the off-chain pattern execution to the on mainchain execution, in addition to the returned parameters, the results produced by the off-chain computation that were saved in a cache are collected and are provided to the attestation procedure executed on the mainchain. Furthermore, once the results are approved by attestation, they are recorded on the mainchain. Our default attestation is to deliver the results of sidechain processing to each of the participants collaborating on the sidechain processing, including the hash of the results, and receive from each actor the digitally signed hash-code of the result (signed by the private key) and thus signifying that the actor accepts the results. If the results are not accepted by an actor, an exception is raised. Of course, the default attestation method may be replaced by an attestation method provided by the developer.

² It should be noted that our statements made about privacy were of general nature and that there are blockchains that do provide for privacy. For instance, the Hyperledger Fabric enables defining certain data to be private and not visible to all participants.

D. Sidechain Processing Control

As the sidechain processing may be used for either cost reduction or for privacy, our tool, described in the next section, delegates the responsibility, of determining which patterns to process on a sidechain, to the modeler/developer. We display the list of LSI patterns of the smart contract, DE-HSM sub-models, to the developer who determines which pattern is or is not processed on a sidechain. For informed decision-making regarding the cost on mainchain vs sidechain, we deploy the smart contract with and without the pattern executing on a sidechain and present both costs to the developer to assist in the decision-making. An example will be provided in the next section that describes the TABS tool.

VI. TABS TOOL AND EVALUATION

We developed a tool, called Transforming Automatically BPMN Models into Smart Contracts (TABS) as a PoC that our approach is feasible and to discover any issues that may arise when a designed system undergoes implementation. We first describe the tool and then its evaluation.

A. TABS Tool for Transforming Automatically BPMN Models into Smart Contracts

Transformation starts with a BPMN model specification expressed in XML as per BPMN specifications [36]. The TABS tool invokes Camunda software platform [35] that is used by the developer to create a BPMN model and store it in XML format. Fig. 11 shows a partial screen of the tool when creating a BPMN model for the supply-chain application. Once the modeler creates a BPMN model and stores it in an XML format, it is used as input to transformations using tabs appearing under the title. The first two tabs deal with BPMN modeling, one for a BPMN choreography, and one for a model expressed in core BPMN elements. Note that the term choreography here refers to the BPMN concept of choreography that shows “conversations” that eventually need to be elaborated upon by the modeler into a model expressed using BPMN’s core elements. Thus, the term “choreography”, as used in BPMN, is not to be confused with the term choreography as used in previous work on transforming BPMN models into smart contracts, in which choreography refers to the choreography of processes into which a BPMN model is transformed and wherein the choreography is a major part of the job mediator’s job of the system architecture of Fig. 2.

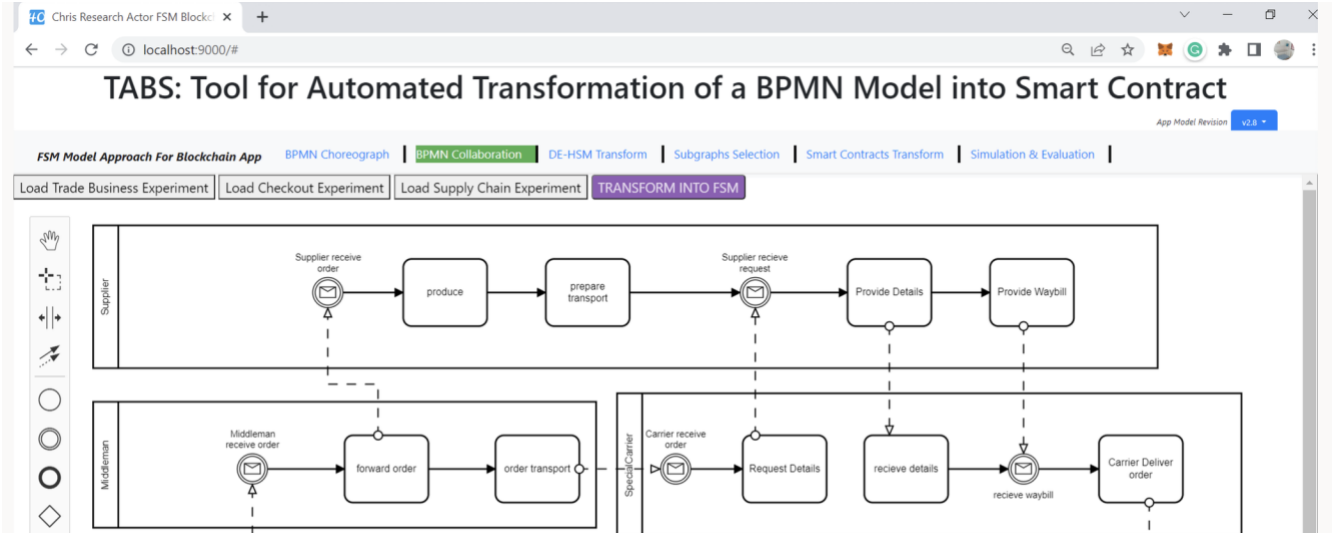


Fig. 11. TABS tool invoking Camunda platform to create a BPMN model of the supply-chain application.

Once the BPMN model is developed, the modeler’s interaction with the tool consists of (i) guiding the transformation of the BPMN model into a smart contract using the tabs and (ii) supplying the template methods, for BPMN tasks, with code. We facilitate creation of a smart contract and sidechain smart contract(s) either for Ethereum or Hyperledger fabric blockchains. However, the tool does not facilitate automated creation of the underlying blockchain fabric itself. In fact, the tool supports using two blockchain types: (i) Hyperledger fabric and (ii) Ethereum or EVM-based blockchain such as Quorum. Either type can be used for the mainchain or for a sidechain(s), wherein the mainchain smart contract invokes methods of the smart contract deployed on a sidechain.

Fig. 12(a) shows a screenshot after the supply-chain BPMN model, shown in Fig. 3, was transformed into the DE-HSM model having the four LSI subgraphs. It shows the BPMN graph and its LSI subgraphs that have been identified by the TABS tool. Fig. 12(b) was not generated by the tool but was included to show the BPMN representation of the corresponding LSI subgraphs of Fig. 12(a).

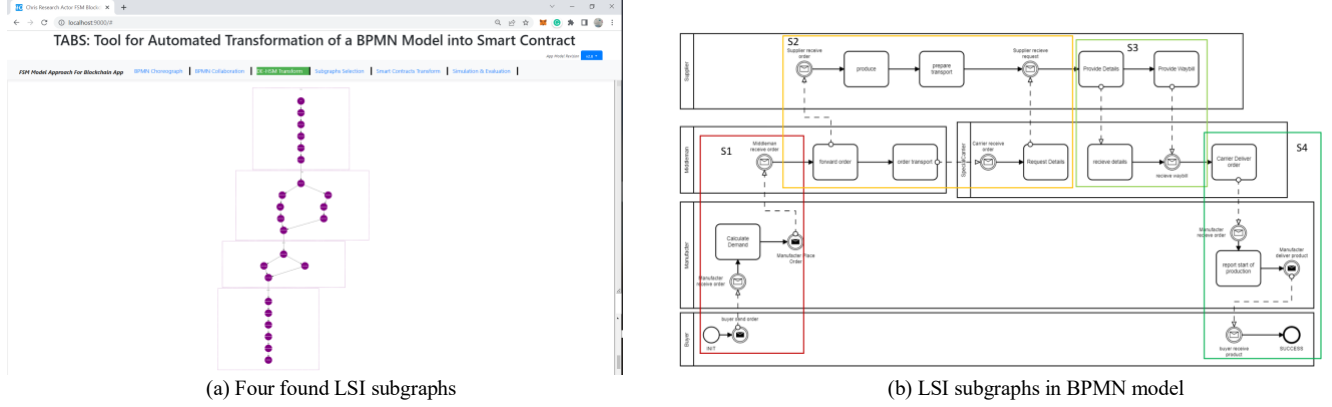


Fig. 12. LSI subgraph after transformation into a DE-HSM model

The modeler/developer is also involved in the decision-making on which of the independent subgraph patterns should be deployed on a sidechain in a form of a separate smart contract that interacts with the main contract deployed and executed on the main blockchain. The modeler then selects which of the Ethereum or Hyperledger fabric is to be used for each of the mainchain and a sidechain – for testing purposes we have local blockchains, one for each of the Ethereum and Hyperledger fabric. On Hyperledger fabric, we also have channels prepared that are used for deployment of smart contracts for both the mainchain and sidechains. For sidechains compatible with Ethereum, we use Quorum sidechain.

Once the selection is made by the developer, the model is transformed into the methods of a smart contract(s) that are deployed and executed. The modeler can examine the generated system by stepping through execution message-by-message, while the tool shows the progress graphically by showing the change to states of the individual FSMs representing the DE-FSM sub-models. This feature is helpful in testing and manual verification. Delays are also shown as execution proceeds step-by-step.

To evaluate the latency and cost of the overall execution, the tool facilitates execution of each method while measuring its latency and cost. It should be noted that we show the cost of execution in Wei (or GWei) for Ethereum or EVM based blockchains/sidechains. Latency and costs are calculated for two cases: (i) When all methods of smart contract are deployed on the mainchain only, and (ii) when execution proceeds on the mainchain with the selected patterns deployed on a sidechain. Thus, the developer can compare the latency and costs when execution is on the mainchain only and when the selected patterns are processed on a sidechain. Fig. 13 exhibits a screenshot showing the cost of execution on a local Ethereum mainchain and a local Quorum sidechain, where the cost of execution is in GWei and is derived by adding the cost of individual Solidity instructions of the smart contract methods using web3.js [44]. As far as cost is concerned, for Ethereum we estimate the cost in GWei units, using web3.js, which is certainly useful to the modelers. Cost for the case when Hyperledger fabric is used depends directly on the cost of the underlying blockchain infrastructure and we have not tackled as yet how to derive it. As in the case of the public Ethereum blockchain, blockchain as a service for Hyperledger fabric is not cheap and it depends on the blockchain configuration in terms of the network nodes, network bandwidth, etc.

B. Evaluation

As mentioned above, we developed the TABS tool as a PoC that our approach is feasible and to explore its properties, such as latency and cost. We first outline our evaluation method and then discuss the use cases we tested, transformation correctness, latency, and cost.

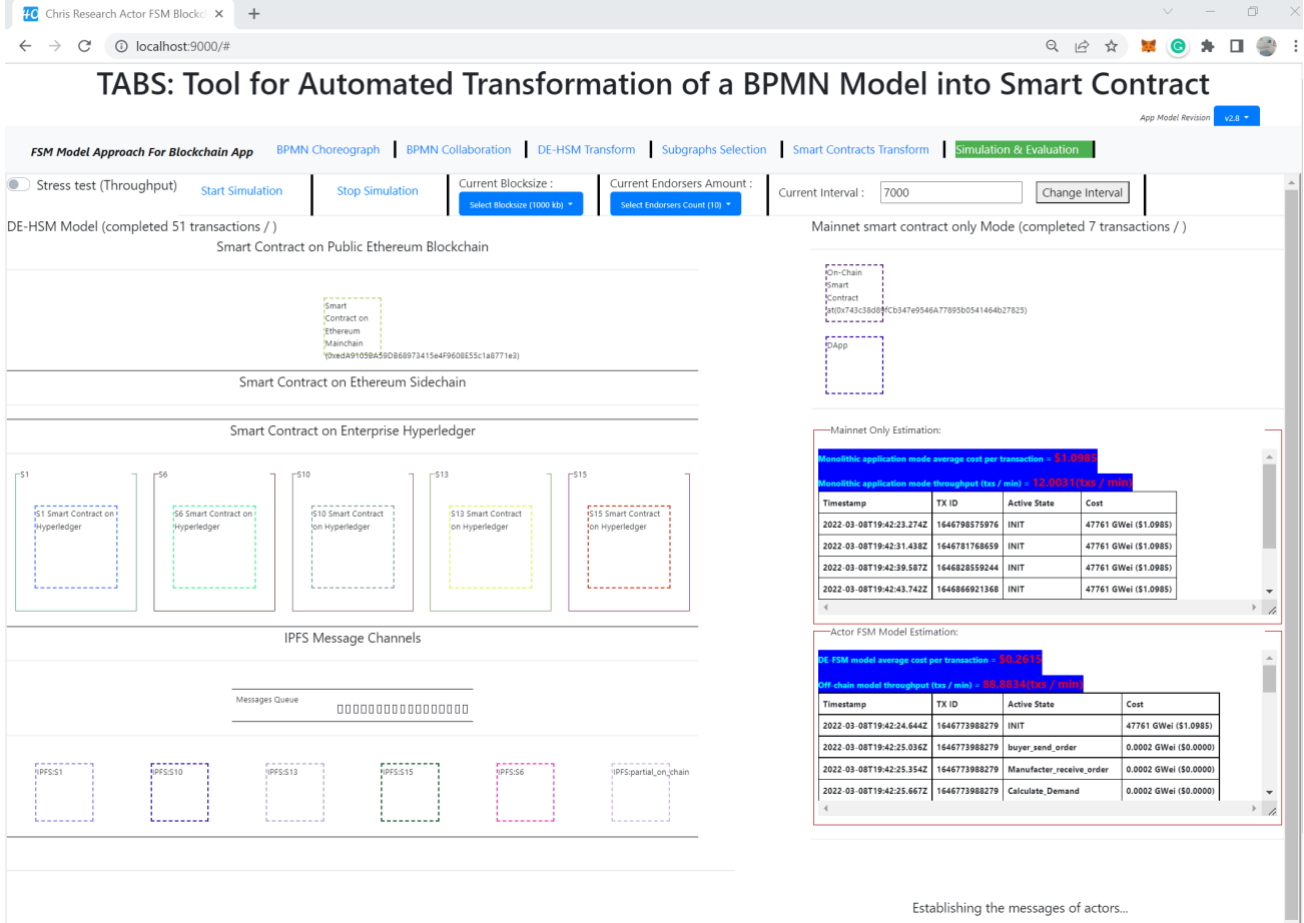


Fig. 13. Stress testing smart contract methods execution of bulk business processes

1) Evaluation method

The TABS tool is a typical two-tier application. The frontend was mainly written in NodeJS, which focuses on the tasks such as:

- Establish distributed message channel with backend IPFS nodes.
- Provide the canvas to the user for composing the BPMN diagram; This was implemented by BPMN.io [45].
- Transform the BPMN diagram to DE-HSM mode; This part was powered by GraphViz JS library [46].
- Provide the Ethereum Solidity abstract interfaces and Hyperledger NodeJS abstract interfaces for smart contracts transformation; Their interfaces were coded in Solidity and NodeJS, respectively [47], [48].
- Compile the Smart Contract with Ethereum powered by NodeJS Worker Threads [49].
- Compile the Smart Contract with IBM Blockchain Platform Developer Tools [50].
- Deploy the Ethereum Smart Contract with EtherJS and Web3.js [51][44].
- Deploy the Hyperledger Chaincode with Microfab API [52].
- Establish the IPFS message channel with IPFS API [53].
- Implement evaluation and simulation process with NodeJS [54].

We use three cloud servers provided by DigitalOcean cloud [55]. Each server is equipped with 2 CPUs, 4GB memory and 80GB disk space. The operating system running on the servers is Ubuntu. We installed, configured, and ran Ganache-CLSI on each server with different parameters to emulate a realistic working environment. The mainchain blockchain was configured with different parameters for block time, endorsers, and block size, but with values that are close to the public Ethereum blockchain, while the sidechain networks were configured as Quorums for Ethereum smart contract and Microfab for Hyperledger Chaincode. Moreover, our TABS tool allows testers to deploy smart contracts on actual public Ethereum and Ropsten sidechain as long as users have sufficient cryptocurrency on those blockchains [56]. Furthermore, each server is an IPFS node located in a preconfigured IPFS private cluster so that each participant has dedicated access to the corresponding IPFS node and data space.

The evaluation process of a use case starts with BPMN modelling. At the end of modelling, TABS automatically transforms the BPMN model to a DE-HSM model during the design phase. The modeler then provides the code for the tasks of the BPMN elements. The DE-HSM model is presented to the modeler together with information on the cost of execution on the mainchain only and on the mainchain with sidechain processing of the selected subgraphs. The modeler uses the information presented to select which of the LSI subgraphs are to be deployed and processed on a sidechain and then chooses which of the Ethereum Solidity and Hyperledger chain-code are to be used for the smart contract generation on the mainchain and a sidechain. The generated code for the smart contract(s) may be reviewed by the developer.

Once the compiled smart contracts are deployed on blockchain networks, the modeler can choose which evaluation process should be used for the run-time phase. The modeler can observe invocation of the smart contract methods. The TABS tool allows the modeler to evaluate the deployed model by an input stream that simulates expected input from the DApp as it would be submitted by the DApp to the DApp interface. Measured are delays per smart contract call, for completion of executing a sub-model, or per execution of the DApp application (on BPMN diagram, from the start event to the end event).

2) Use Cases

For our evaluation purpose, we used three different use cases.

- i. *Supply Chain*: This use case was adopted from [16] and was discussed throughout this paper as a running example (see figures 3 and 7). The model has ten tasks and four gateways. Supply chain use case begins with the buyer issuing new order. Once the manufacturer receives the order, she/he calculates demand and places an order with a middleman. Middleman concurrently sends the order to the supplier and orders transport from carrier. The producer fabricates the product and prepares it for transport. Carrier, upon receiving the request from the middleman, requests details from the supplier. Supplier provides the details to the carrier and then it prepares and provides the waybill to the carrier. Upon receiving details about the product and the waybill, both from the supplier, the carrier delivers the order to the manufacturer. Upon receiving the order, the manufacturer starts the production and when that is finished, it delivers the product to the buyer, who receives the order.
- ii. *Order Process*: This process is adopted from [57] and its BPMN diagram is shown in Fig. 14(a). It is a relatively straightforward use case with five tasks, and two gateways. The use case starts with the customer preparing an order that is delivered to order handling. Order handling checks the order and then concurrently (i) confirms the order with the customer and (ii) initiates shipment preparation by the shipper. Shipper prepares the order for shipment and then sends it to the customer. The customer, upon receiving the order confirmation and the order from the shipper, accepts the order, which completes the workflow. Fig. 14(b) shows the BPMN diagram with the three found LSI subgraphs identified by the colored rectangles.

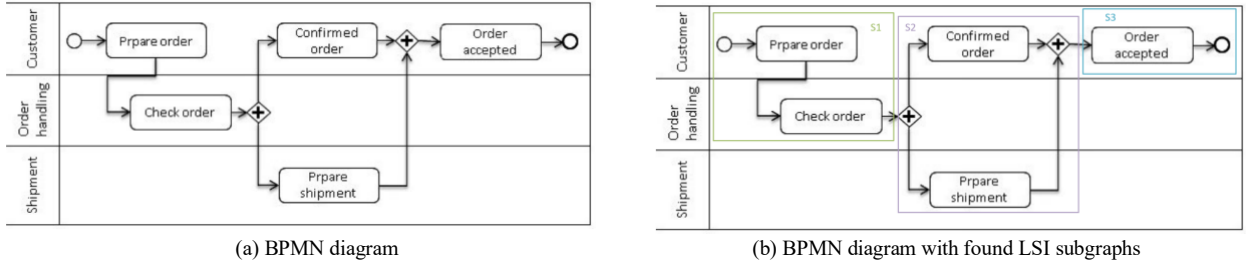


Fig. 14. BPMN diagram for the Order Proces use case (adopted from [58])

- iii. *Trade*: We reported on this use case in some detail in our previous research [42]. The use case was adopted from [58] and it contains 19 tasks, and 5 gateways. As the BPMN diagram of the application would be too busy, we show in Fig. 15 its state diagram representation with some of the independent subgraphs also shown. The model shows that the product is posted for sale and after offer is made, the buyer and seller negotiate price. Once an agreement is reached on the price, a contract is prepared and signed that stipulates escrow deposit and the matter of delivery. Once the buyer makes a deposit to an escrow account, the shipment proceeds to port. The shipment includes crossing borders and hence involves customs. Afterwards, the shipment is loaded on a ship, delivered to a port, and then processed at the destination customs. It is then unloaded from ship to port, picked up by the buyer, payment terms are executed, and finally escrow deposit is returned.

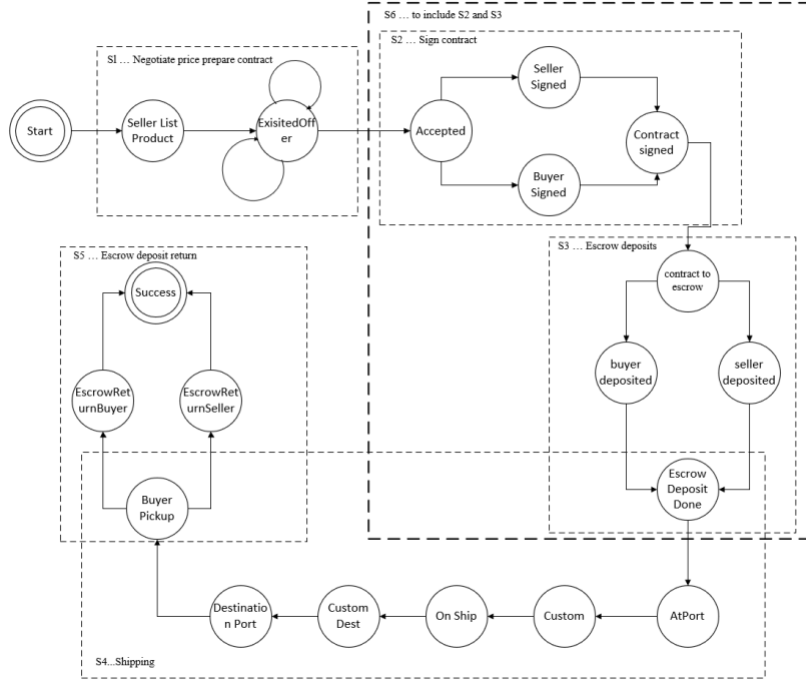


Fig. 15. State transition diagram for the Trade Business use case (adopted from [43])

3) Latency and cost

We now report on evaluation of the three use cases in terms of latency and cost. However, the cost is derived only for Ethereum using web3.js [44] that estimates the cost of each instruction in a smart contract in Wei. In this section we simply report observed values as we are unable to do comparison with the other reported work because hardware and software configurations and environments vary greatly. We report observed values for each case when Ethereum is used and when the Hyperledger fabric is used. We comment on our observations when sidechains are used for off-chain processing in the next subsection.

a) Latency

The TABS tool can measure various latencies, ranging from one FSM state transition, through latency for executing a sub-model or a smart contract method, to latency for the whole DApp. Fig. 16 shows the delays in milliseconds for each use case execution and for each of Ethereum and Hyperledger fabric. Due to the more complex consensus algorithm, Ethereum exhibits much higher delays than Hyperledger.

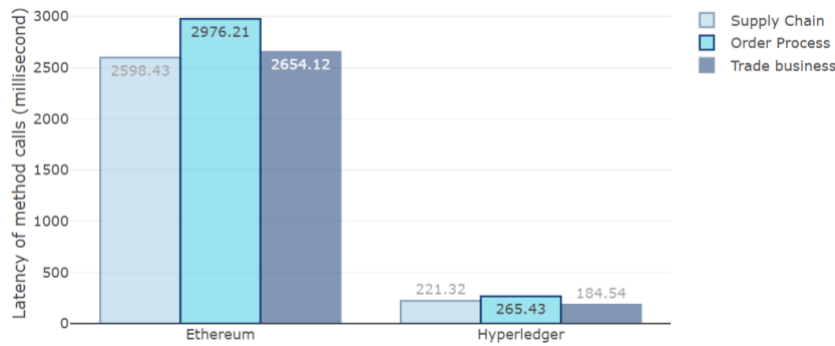


Fig. 16. Latency for each of the three use cases for Ethereum and for Hyperledger fabric

b) Cost

Costs per method and the overall cost can also be estimated automatically through TABS by using web3.js [44] that enables calculation of the cost for each instruction. The cost in GWei for each of the three use cases is shown in Table 1 for each of the three use-cases. The table also shows the cost, in USD at rates prevailing at the time of measurements, of one application

workflow of each use case for the public Ethereum. As the Hyperledger fabric is a private blockchain, the cost is primarily due to hardware and network costs.

Table 1. Cost of processing for Ethereum environment

	Gas (GWei)	Cost (USD)
Supply Chain	95,854	0.2615
Order Process	57,512	0.1569
Trade Business	182,195	0.4945

4) *Correctness (non-conforming vs conforming traces)*

In published research on transforming BPMN models to smart contracts, reporting was done on the correctness of the smart contract by recognizing conforming and non-conforming traces, wherein a trace is input stream from the DApp. Conforming trace represents input from the DApp that is correct in terms of correctness of input provided by the DApp at appropriate points of a smart contract execution. The smart contract should recognize incorrect input provided by the DApp. We tested the smart contract implementation and reported that 100% accuracy was achieved in recognizing conformant and non-conformant input sequence/trace. This is not surprising - after all, the smart contract is built through DE-HSM modelling that transforms the BPMN model into a DE-HSM model that, in essence, is formed by a 1-to-1 mapping between the BPMN model and the DE-HSM model, as a collection of DE-FSM sub-models, wherein each sub-model corresponds directly to the BPMN diagram when viewed as a DAG. Thus, in the absence of errors in the TABS tool, the recognition of correct and incorrect sequencing in input should be 100%.

5) *Sidechain processing*

One of the distinguishing features of our approach is that we can move blockchain patterns, represented as an individual DE-HSM sub-models, to sidechain for either of the following two purposes or their combination: (a) reducing the cost of processing and (ii) privacy.

a) *Sidechains for cost reduction*

Deploying patterns for processing on a sidechain for the purposes of reducing the cost of processing is useful only when sidechain processing is cheaper in comparison to the mainchain processing. This is important for any smart contract targeted for deployment on a blockchain – blockchains are expensive relative to centralized systems. As an example, processing on the public Ethereum blockchain is much more expensive than on the Quorum chain and hence sidechain processing is attractive for reducing the cost. For Hyperledger fabric, using channels as sidechains for cost reduction does not make sense as a mainchain on a Hyperledger is just one of the many channels. Hyperledger fabric may be used as the mainchain while some other blockchain is used for a sidechain, as long as it (sidechain blockchain) is cheaper than the mainchain.

For each use case, we calculated the cost of processing one execution of a smart contract. For each use case, Fig. 17 shows the cost of processing on the Ethereum mainchain only and then when a Quorum sidechain is used with the mainchain. When sidechain processing is used, for each use case we select all sub-models to be processed on a sidechain. The selected sub-models are identified by either colored or dashed rectangles in Fig. 7, 12, and 15. Fig. 19 shows that, due to Quorum cost of processing being much lower than the cost of processing on the mainchain – processing off-chain is very advantageous.

We also compare the latency of processing, which is shown in Fig. 18, for the mainchain processing on Ethereum and for mainchain with sidechain processing on Quorum for each of the use cases. Latency is also reduced by sidechain processing. Similarly, we used sidechain processing for Hyperledger fabric by deploying the main smart contract on one Hyperledger channel and a sidechain on another channel of the same Hyperledger fabric blockchain. Not surprisingly, latency with sidechain processing is higher than just processing on the mainchain only, as is shown in Fig. 19. Sidechain processing on a Hyperledger is achieved by using another Hyperledger channel that incurs the same cost as processing on the mainchain, which is also a Hyperledger channel. Due to overhead to facilitate sidechain processing, latency is higher when sidechain is used.

a) *Sidechains for privacy*

As discussed already, sidechains may also be used for privacy. In more complex smart contracts, such as the *trade* use case, certain parts of the contract should not be visible to all participants of the smart contract. For instance, after the buyer and seller agree on the price, the price should not be visible to some of the other actors. By choosing to deploy and execute an independent subgraph pattern, i.e., a sub-model, on a sidechain, privacy is also provided as only the actors who participate in the sidechain sub-model execution can observe all data pertaining to that sidechain processing, while the other actors do not have access to that data. However, all actors have access to the results of computation that are stored on the mainchain. As was already noted before, the above statements are subject to privacy features of individual blockchains as some blockchains, such as Hyperledger Fabric, do support private data on a channel, private data that is not visible to other participants.

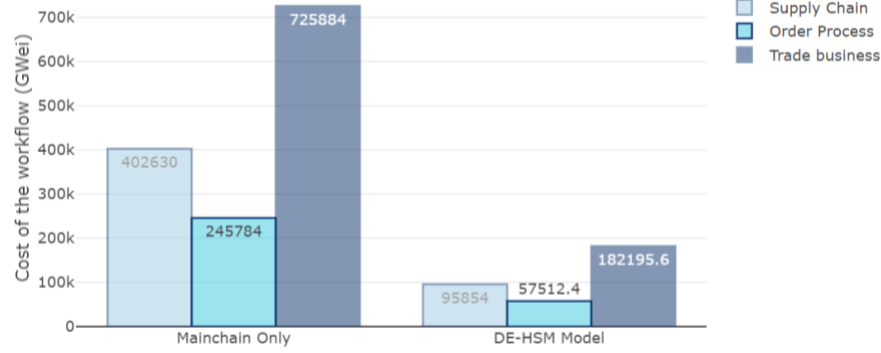


Fig. 17. Ethereum cost of processing on mainchain only vs processing on mainchain with sub-models on a sidechain.

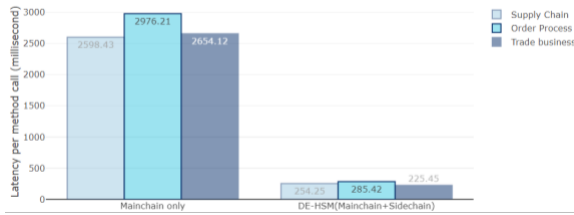


Fig. 18. Ethereum latency of processing on mainchain vs sub-models on Quorum sidechain.

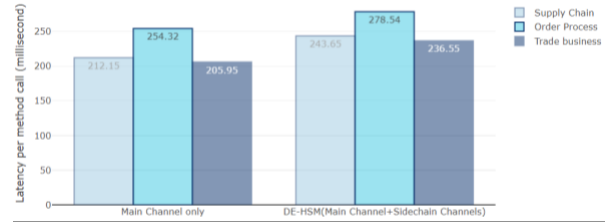


Fig. 19. Hyperledger latency of processing of all on one channel vs processing sub-models on another channel

VII. CONCLUDING REMARKS

This paper provides contributions to the research on generating smart contracts from a BPMN model. As in previously reported research, we minimize the modeler's interaction with the system in that once the BPMN model is input, the modeler simply guides the tool in generation of the smart contract methods. The modeler guides the tool in the transformation process and provides necessary information to the tool, such as the code for template methods implementing BPMN task elements or information on which parts of the smart contract should be deployed on a sidechain. In addition, the modeler/developer can use the tool to explore execution properties of the methods of the smart contract. Currently, our tool provides creation and deployment of contracts for either Ethereum or Hyperledger blockchains.

Although the overall architecture of our system is in many respects similar to work of others, the following two are the major features that are very different from the work reported by others.

- We analyze the BPMN diagram and use DE-HSM modeling to identify patterns, referred to as LSI independent subgraphs, that are localized in that once execution of the pattern starts, it remains local to the pattern until there is an exit from that pattern. Furthermore, each pattern has only one "entry" and one "exit" point. Such patterns naturally lend themselves for decomposition purposes that our approach exploits. We use such patterns for forming the sub-models, wherein the functionality of each sub-model is represented by an FSM or concurrent FSMs. Choreography (not a BPMN term) of processes, that is workflow determining which sub-models are executed in which sequence, is guided by the interconnection amongst the sub-models, while functionality of each sub-model is represented by an FSM, or possibly a number of concurrent FSMs. The interconnection of the sub-models is represented by the DE-HSM model. Consequently, as the DE-HSM is a model that is an equivalent representation of a BPMN model, correctness of representing the BPMN model using the DE-HSM model is assured.
- We provide for transformation of sub-models, selected by the modeler, into a separate contract that is deployed and executed on a sidechain, while the main smart contract interacts with the DApp and invokes these sidechain methods as is appropriate. Sidechain processing may be chosen by a modeler (i) to reduce the cost of processing if sidechain processing is cheaper than processing on a mainchain (e.g., processing on a Quorum sidechain in conjunction with the main contract being deployed on the public Ethereum blockchain); or (ii) to support privacy by processing a pattern on a sidechain.

We made several simplifying assumptions on the BPMN model, such as limited looping/repeating or parallel subprocesses, and no feedback loops in the BPMN model that cannot be expressed as looping or concurrent subprocesses, that need to be

addressed. The tool's limitation is that, prior to the execution of a looping subprocess, the number of iterations needs to be known. The assumption on the parallel subprocesses is more restrictive in that the maximum number of parallel streams to execute a parallel sub-process needs to be known at the time of transforming the BPMN model into the methods of a smart contract. We are investigating approaches to reduce these limitations.

The BPMN standard defines a transaction on a sub-process, in which case the whole subprocess must complete or any activity of the subprocess must be undone by a compensating transaction if the subprocess is unable to complete successfully. As was already discussed before, a blockchain also has a transaction that is defined as any set of ledger updates made by an execution of a *single smart contract method*. However, a trade transaction is long-term and may include many activities, activities performed by a subset of the application participants on a subset of data used by the application. And these three different concepts do not align. How to support the long-term trade transactions, which span a number of calls to the smart contract methods, is still an open question that we are currently addressing. There are also some of the BPMN elements, such as error, escalation, and multiple and multiple parallel attributes that our TABS system does not support, but we are working on their incorporation into the system.

Currently we provide a demo and limited-term online access to the tool upon request that can be made by emailing Chris.Liu@dal.ca. In near future, we plan on preparing appropriate documentation and including the code in a repository to facilitate future potential collaborations.

Of course, our current implementation is only a proof-of-concept, and many issues still need to be addressed. For instance, as contract execution consists of removal of events from Q and processing them, issues of efficiency and scalability need to be addressed. The use of a global logical clock to timestamp events, which are stored in the timestamp-priority ordered queue (Q), may also lead to scalability issues that need to be investigated. We note that the properties of trade applications are such that the speed of processing is not a priority as the transactions in this vertical are relatively long-term. The issue of scalability may arise due to the increase in the complexity of applications and in the number of applications that need to be supported.

CONFLICT OF INTEREST

The authors declare no conflict of interest.

AUTHOR CONTRIBUTIONS

All three authors participated in research and writing of this paper.

REFERENCES

- [1] J. Eberhardt, S. Tai, 2017. On or Off the Blockchain? In: De Paoli F., Schulte S., Broch Johnsen E. (eds) Service-Oriented and Cloud Computing. ESOC 2017. Lecture Notes in CS, 3-15, vol 10465, Springer.
- [2] J. Poon, V. Buterin, 2017. Plasma: Scalable Autonomous Smart Contracts. <https://plasma.io/plasma.pdf> on Nov. 12, 2020. 47 pages.
- [3] D. Yang, C. Long, H. Xu, S. Peng, 2020. A Review on Scalability of Blockchain. In *Proceedings of the 2020 The 2nd International Conference on Blockchain Technology (ICBCT'20)*. Association for Computing Machinery, New York, NY, USA, 1–6. DOI:<https://doi.org/10.1145/3390566.3391665>
- [4] Shard Chains | Ethereum.org. [ethereum.org. https://ethereum.org/en/eth2/shard-chains/](https://ethereum.org/en/eth2/shard-chains/), 2021 (access 26 March 2022).
- [5] P. J. Taylor, T. Dargahi, Dehghantanha, R. M. Parizi, 2019. A Systematic Literature Review Of Blockchain Cyber Security - ScienceDirect. A systematic literature review of blockchain cyber security - ScienceDirect. <https://www.sciencedirect.com/science/article/pii/S2352864818301536>.
- [6] S. Khan, F. Loukil, C. Ghedira-Guegan, E. Benkhelifa, A. Bani-Hani, 2021. Blockchain smart contracts: Applications, challenges, and future trends. *Peer Peer Netw Appl.* 2021 Apr 18:1-25. doi: 10.1007/s12083-021-01127-0. Epub ahead of print. PMID: 33897937; PMCID: PMC8053233.
- [7] A. Vacca, A. Annalisa et al. 2021. A systematic literature review of blockchain and smart contract development: Techniques, tools, and open challenges. *J. Syst. Softw.* 174 (2021): 110891.
- [8] R. Belchior, A. Vasconcelos, S. Guerreiro, M. Correia. 2021. A Survey on Blockchain Interoperability: Past, Present, and Future Trends. *ACM Comput. Surv.* 54, 8, Article 168, 41 pages. DOI:<https://doi.org/10.1145/3471140>
- [9] K. Saito, H. Yamada, 2016. What's So Different about Blockchain? *Blockchain: Probabilistic State Machine*. 2016 ICDCSW. 168-175.
- [10] J. A. Garcia-Garcia, N. Sánchez-Gómez, D. Lizcano, M. J. Escalona and T. Wojdyński, "Using Blockchain to Improve Collaborative Business Process Management: Systematic Literature Review," in *IEEE Access*, vol. 8, pp. 142312-142336, 2020, doi: 10.1109/ACCESS.2020.3013911.
- [11] C. Lauster, P. Klinger, N. Schwab, F. Bodendorf, 2020. Literature Review Linking Blockchain and Business Process Management in 15th International Conference on Wirtschaftsinformatik, March 08-11, 2020, Potsdam, Germany. https://doi.org/10.30844/wi_2020_r10-klinger
- [12] O. Levasseur, M. Iqbal, R. Matulevičius, 2021. Survey of Model-Driven Engineering Techniques for Blockchain-Based Applications. PoEM'21 Forum: 14th IFIP WG 8.1 Working Conference on the Practice of Enterprise Modelling,

- [13] P. Tolmach, Y. Li, S. Lin, Y. Liu, Z. Li. 2021. A Survey of Smart Contract Formal Specification and Verification. *ACM Comput. Surv.* 54, 7, Article 148 (September 2022), 38 pages. DOI:<https://doi.org/10.1145/3464421>
- [14] S. Bragagnolo, H. Rocha, M. Denker, S. Ducasse, 2018. SmartInspect: Solidity smart contract inspector. In: 2018 International workshop on blockchain oriented software engineering (IWBOSE), pp 9–18
- [15] J. Mendling, I. Weber, W. V. D. Aalst, J. V. Brocke et al., 2018. Blockchains for Business Process Management - Challenges and Opportunities. *ACM Trans. Manage. Inf. Syst.* 9, 1, Article 4 (March 2018), 16 pages. DOI:<https://doi.org/10.1145/3183367>
- [16] I. Weber, X. Xu, R. Riveret, G. Governatori, A. Ponomarev, J. Mendling, 2016. Untrusted Business Process Monitoring and Execution Using Blockchain. *Proceedings of Business Process Management Conference, Rome, Italy, 2016*, 329–347.
- [17] O. López-Pintado, L. García-Bañuelos, M. Dumas, I. Weber, A. Ponomarev, 2019. Caterpillar: a business process execution engine on the Ethereum blockchain. *Softw. Pract. Exper.* 2019; 49: 1162–1193. <https://doi.org/10.1002/spe.2702>
- [18] A. B. Tran, Q. Lu, I. Weber, 2018. Lorikeet: A Model-Driven Engineering Tool for Blockchain-Based Business Process Execution and Asset Management. 16th International Conference on Business Process Management, BPM 2018, Sydney, Australia, September 9–14, 2018, pp. 56–60.
- [19] O. López-Pintado, M. Dumas, L. García-Bañuelos, I. Weber, 2019. Dynamic role binding in blockchain-based collaborative business processes, in: *Advanced Information Systems Engineering - 31st International Conference, CAiSE 2019, Rome, Italy, June 3–7, 2019, Proceedings, 2019*, pp. 399–414.
- [20] O. López-Pintado, M. Dumas, L. García-Bañuelos, I. Weber, 2022. Controlled flexibility in blockchain-based collaborative business processes. *Inf. Syst.*, 104, 101622.
- [21] C. Di Ciccio, A. Cecconi, M. Dumas et al., 2019. Blockchain Support for Collaborative Business Processes. *Informatik Spektrum* 42, 182–190 (2019). <https://doi.org/10.1007/s00287-019-01178-x>
- [22] F. Loukil, K. Boukadi, M. Abed, C. Ghedira-Guegan, 2021. Decentralized Collaborative Business Process Execution Using Blockchain. *World Wide Web*, Springer Verlag, 2021, (10.1007/s11280-021-00901-7). (hal-03250846)
- [23] Q. Lu, A. B. Tran, I. Weber, et al. Integrated model-driven engineering of blockchain applications for business processes and asset management. *Softw. Pract. Exper.* 2021; 51: 1059–1079. <https://doi.org/10.1002/spe.2931>
- [24] Spalazzi, L., Spegni, F., Corneli, A., Naticchia, B. Blockchain based choreographies: The construction industry case study. (2021) *Concurrency and Computation: Practice and Experience*
- [25] A. Mavridou, A. Laszk, 2018. Designing Secure Ethereum Smart Contracts: A Finite State Machine Based Approach. *Financial Cryptography*. Springer-Verlag, Berlin, Heidelberg, 523–540. DOI:https://doi.org/10.1007/978-3-662-58387-6_28
- [26] A. Mavridou, A. Laszka, 2018. Tool Demonstration: FSolidM for Designing Secure Ethereum Smart Contracts. In: Bauer L., Kusters R. (eds) *Principles of Security and Trust. POST 2018. Lecture Notes in Computer Science*, vol. 10804. Springer. 270–277.
- [27] D. Harel (1987). Statecharts: a visual formalism for complex systems. *Science of Computer Programming*, Volume 8, Issue 3, 1987, Pages 231–274, ISSN 0167-6423. DOI: [https://doi.org/10.1016/0167-6423\(87\)90035-9](https://doi.org/10.1016/0167-6423(87)90035-9).
- [28] A. Girault and B. Lee and E.A. Lee (1999). Hierarchical Finite State Machines with Multiple Concurrency Models. *IEEE Transactions on Computer-Aided Design*, June 6, 1999. 742–760
- [29] M. Yannakakis (2000) Hierarchical State Machines. In: van Leeuwen J., Watanabe O., Hagiya M., Mosses P.D., Ito T. (eds) *Theoretical Computer Science: Exploring New Frontiers of Theoretical Informatics. TCS 2000. Lecture Notes in CS*, vol 1872. Springer. 315–330.
- [30] C. A. R. Hoare, 1978. Communicating sequential processes. *Commun. ACM.* vol. 21, no. 8.
- [31] C. Cassandras, 1993. *Discrete Event Systems, Modeling and Performance Analysis*. Irwin: Homewood IL.
- [32] C. Liu, P. Bodorik, D. Jutla. 2021. Blockchain Privacy, Scalability, and Separation of Concerns Using Multi-modal Modeling. Accepted by Fourth International Conference on Blockchain Technology and Applications (ICBTA'21), December 17th–19th, 2021. ACM, New York, NY, USA, 15 pages. <https://web.cs.dal.ca/~gang/ICBTA/>
- [33] L. Dikmans, 2008. Transforming BPMN into BPEL: Why and How. <https://www.oracle.com/technical-resources/articles/dikmans-bpm.html> (access 26 March 2022).
- [34] R.M. Dijkman, M. Dumas, C. Ouyang, 2008. Semantics and analysis of business process models in BPMN. In *Elsevier Information and Software Technology Journal*, 50 (2008), pp. 1281–1294.
- [35] Camunda Modeler conformance to BPMN and BPEL - Modeler - Camunda Platform Forum. (2016, April 14). Camunda Platform Forum; forum.camunda.org. [https://doi.org/Camunda Modeler conformance to BPMN and BPEL - Modeler - Camunda Platform Forum](https://doi.org/Camunda%20Modeler%20conformance%20to%20BPMN%20and%20BPEL%20-%20Modeler%20-%20Camunda%20Platform%20Forum)
- [36] BPMN 2.0 Introduction · Flowable Open Source Documentation. (n.d.). BPMN 2.0 Introduction · Flowable Open Source Documentation; www.flowable.com. Retrieved March 29, 2022, from <https://www.flowable.com/open-source/docs/bpmn/ch07a-BPMN-Introduction>
- [37] BPMN 2.0 Symbols - A complete guide with examples. - Camunda. (2021, October 28). Camunda; [camunda.com](https://camunda.com/bpmn/reference/). <https://camunda.com/bpmn/reference/>
- [38] Object-relational impedance mismatch - Wikipedia. (2020, August 1). Object-Relational Impedance Mismatch - Wikipedia; [en.wikipedia.org](https://en.wikipedia.org/wiki/Object%E2%80%93relational_impedance_mismatch). https://en.wikipedia.org/wiki/Object%E2%80%93relational_impedance_mismatch (access 26 March 2022).
- [39] M. Steichen, B. Fiz, R. Norvill, W. Shbair and R. State, Blockchain-Based, Decentralized Access Control for IPFS, *2018 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)*, 2018, pp. 1499–1506, doi: 10.1109/Cybermatics_2018.2018.00253.
- [40] What Is BPMN? - BPMN-Guide. (n.d.). What is BPMN? - BPMN-Guide. <https://bpmn.gitbook.io/bpmn-guide/what-is-bpmn> (access 26 March 2022).
- [41] C. G. Liu. 2021. FSM for Modeling for Off-Blockchain Computation. Faculty of Graduate Studies Online Theses. Dalhousie University, Halifax, Nova Scotia, Canada. <http://hdl.handle.net/10222/80335>
- [42] P. Bodorik, C. Liu, D. Jutla, 2021. Using FSMs to Find Patterns for Off-Chain Computing: Finding Patterns for Off-Chain Computing with FSMs. In 2021 The 3rd International Conference on Blockchain Technology (ICBCT '21), March 26–28, 2021, Shanghai, China. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3460537.3460565>

- [43] C. Liu, P. Bodorik, D. Jutla, 2021. A Tool for Moving Blockchain Computations Off-Chain. In Proceedings of 2021 3rd ACM International Symposium on Blockchain and Secure Critical Infrastructure (BSCI'21), June 3–7, 2021, Virtual Event, Hong Kong. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3457337.3457848>
- [44] web3.js - Ethereum JavaScript API — web3.js 1.0.0 documentation. (n.d.). Web3.js - Ethereum JavaScript API — Web3.js 1.0.0 Documentation; web3js.readthedocs.io. <https://web3js.readthedocs.io/> (access 26 March 2022).
- [45] bpmn.io · GitHub. 2022. GitHub; github.com. <https://github.com/bpmn-io> (access 26 March 2022).
- [46] Graphviz. (n.d.). Graphviz; graphviz.org. Retrieved March 29, 2022, from <https://graphviz.org/>
- [47] Solidity 0.8.13 documentation. (n.d.). Solidity — Solidity 0.8.13 Documentation; docs.soliditylang.org. Retrieved March 29, 2022, from <https://docs.soliditylang.org/en/v0.8.13/>
- [48] Hyperledger Fabric SDK for Node.js. (n.d.). Fabric-SDK-Node; hyperledger.github.io. Retrieved March 29, 2022, from <https://hyperledger.github.io/fabric-sdk-node/>
- [49] Worker threads | Node.js v17.7.1 Documentation. (n.d.). Worker Threads | Node.js V17.7.1 Documentation; nodejs.org. https://nodejs.org/api/worker_threads.html (access 26 March 2022).
- [50] IBM Cloud Docs. (n.d.). IBM Cloud Docs; cloud.ibm.com. Retrieved March 29, 2022, from <https://cloud.ibm.com/docs/blockchain?topic=blockchain-develop-vscode>
- [51] Documentation. (n.d.). Documentation; docs.ethers.io, from <https://docs.ethers.io>
- [52] IBM-Blockchain. 2022. GitHub - IBM-Blockchain/microfab: Microfab is a containerized Hyperledger Fabric runtime for use in development environments. GitHub; github.com. <https://github.com/IBM-Blockchain/microfab> (access 26 March 2022).
- [53] IPFS cluster consistency model - ipfs-cluster - discuss.ipfs.io. 2019. Discuss.Ipfs.Io; discuss.ipfs.io. [https://doi.org/IPFS cluster consistency model - ipfs-cluster - discuss.ipfs.io](https://doi.org/IPFS%20cluster%20consistency%20model%20-%20ipfs-cluster%20-%20discuss.ipfs.io)
- [54] Node.js. (n.d.). Node.js. Node.js; nodejs.org. Retrieved March 29, 2022, from <https://nodejs.org/en/>
- [55] DigitalOcean – The developer cloud. 2020. DigitalOcean – The Developer Cloud; www.digitalocean.com (access 26 March 2022).
- [56] etherscan.io. (n.d.). TESTNET Ropsten (ETH) Blockchain Explorer. Ethereum (ETH) Blockchain Explorer; ropsten.etherscan.io. Retrieved March 29, 2022, from <https://ropsten.etherscan.io/>
- [57] A. Fleischmann, W. Schmidt, C. Stary. (Re-)Justifying BPM: A Quest for the Interaction Turn Reviewing Subject-Oriented BPM. Business Informatics (CBI), 2013 IEEE 15th Conference. 2013. DOI: 10.1109/CBI.2013.40
- [58] Asgaonkar A. & Krishnamachar B. (2019). Solving the Buyer and Seller's Dilemma: A Dual-Deposit Escrow Smart Contract. IEEE Int. Conf. on Blockchain and Cryptocurrency (ICBC). 262-267.
- [59] Gwei Definition and Explanation [WWW Document], 2022. . Investopedia. URL <https://www.investopedia.com/terms/g/gwei-ethereum.asp> (accessed 4.1.22).
- [60] C. G. Liu, P. Bodorik, D. Jutla, 2021, From BPMN to smart contracts on blockchains: Transforming BPMN to DE-HSM multi-modal model, 7th IEEE International Conference on Engineering and Emerging Technologies (ICEET), 2021, pp. 1-7, doi: 10.1109/ICEET53442.2021.9659771.